

Learning Canonical Register Automata over Ordered Data Domains

Yong Li^{1,2} , Qiyi Tang³ , and Di-De Yen³ 

¹ Key Laboratory of System Software (Chinese Academy of Sciences), Beijing, China

² Institute of Software, Chinese Academy of Sciences, Beijing, China

liyong@ios.ac.cn

³ School of Computer Science and Informatics, University of Liverpool, UK
{qiyi.tang, d.d.yen}@liverpool.ac.uk

Abstract. Register automata are finite automata equipped with memory that recognize data languages over infinite alphabets. In this work, we investigate active learning algorithms for deterministic register automata (DRAs) over *ordered* data domains—covering both dense domains, such as the rationals, and non-dense domains such as the integers. We show that the active learning problem for DRAs over both dense and non-dense ordered domains can be treated within a single unified framework. More specifically, we develop and implement a polynomial-time active learning procedure for DRAs over ordered domains, using oracles for membership, equivalence and memorability queries. The memorability queries were originally introduced for learning DRAs over domains with identity tests. Our unified framework also leads to a new consequence: minimization of DRAs over the non-dense ordered domain of integers is decidable, extending a result previously known only for dense domains. Finally, we give improved complexity bounds of several decision problems for DRAs over ordered domains that are closely related to the queries used in active learning.

1 Introduction

The L^* algorithm, introduced by Angluin [2] within the Minimally Adequate Teacher (MAT) framework, enables a learner to infer an automaton representing an unknown system by interacting with a teacher through two types of queries: membership queries and equivalence queries. This query-based paradigm is often referred to as *active learning* [2, 38]. A membership query asks whether a word u belongs to the target language L , while an equivalence query asks whether a proposed automaton recognizes L . After collecting enough information from membership queries, the learner constructs a conjectured automaton and asks an equivalence query. The teacher either confirms that the conjecture is correct or returns a counterexample, which is then used by the learner to refine the conjecture. This iterative process continues until the learner successfully converges to the target automaton.

Active learning has been widely applied, including learning assumptions for compositional verification [15], software testing [16], detecting errors in network protocol implementations [36], regular model checking [13], extracting automata models for recurrent neural networks [40], verifying binarized neural networks [37], and explaining machine learning models [34].

In this paper, we consider active automata learning for *deterministic register automata* (DRAs) over *ordered* data domains. Register automata (RAs) are automata over infinite alphabets and have been extensively studied [1, 6, 7, 14, 24, 33] motivated by real-world problems where data may have an unbounded domain. An RA stores data symbols in a finite number of registers, which can later be compared against new inputs. RAs are far more expressive than classical finite automata; however, they lose many desirable closure and decidability properties—language inclusion and equivalence become undecidable for nondeterministic RAs [24]. Fortunately, these problems are decidable for DRAs [33]. A key challenge in learning DRAs is that the learner must simultaneously reason about control-state behavior and data constraints, unlike in classical active learning.

In particular, comparisons between input data and register values come in two forms: (1) identity tests, which check equality; and (2) ordered comparisons, which in addition to equality also test whether the input is greater or smaller than a stored value. Most existing work focuses on DRAs restricted to identity tests [9, 23, 24, 33], which limits their ability to capture ordering-based properties. For instance, checking whether an input sequence is sorted (i.e. in non-decreasing order) is recognizable by a DRA with ordered comparison but *not* by any RA using identity tests only. There has been prior work [10, 12] on learning DRAs over ordered domains; however, it relies on tree-based queries over abstract symbolic words, whereas our approach operates directly on concrete words using standard membership queries together with memorability information.

We focus on a family of DRAs introduced in [7] that support ordered comparisons. These automata prohibit duplicate register values and, after each transition, permute registers so that their ordering reflects the order of last occurrence of their stored data. The languages recognized by such DRAs—often called *regular data languages*—admit a Myhill–Nerode-style characterization and have a canonical minimal representation that is simultaneously minimal in both the number of states and the number of registers [7, 25]. This property is crucial for active learning, which relies on a canonical representation of the target formalism, analogous to the Myhill–Nerode theorem [30, 32] that guarantees a unique minimal DFA for regular languages.

A subtle issue arises when comparing dense data domains (e.g. $\mathbb{Q}$) and non-dense domains (e.g. $\mathbb{Z}$). For RAs restricted to identity tests, this distinction is irrelevant. However, for DRAs with ordered comparisons, non-dense domains introduce additional complications.

Contributions. We show that minimal DRAs over $\mathbb{Q}$ and minimal DRAs over $\mathbb{Z}$ that recognize the same language when restricted to integers are structurally equivalent. Consequently, the decidable complexity result of the minimization algorithm of [25], originally developed for dense domains, extends to non-dense

domains as well. Accordingly, we obtain a unified active learning framework that applies uniformly to both dense and non-dense data domains. As a related result, we establish that the configuration reachability problem over $\mathbb{Z}$ is decidable. Based on the canonical form of DRAs, we develop an active learning algorithm for DRAs over *ordered* data domains. This algorithm uses *polynomially* many membership, equivalence and *memorability* queries, which precisely identify which symbols must be stored in registers when processing a data word⁴. The main technical challenge is that, unlike learning DRAs with identity tests [9, 23], which rely on equality-based reasoning to capture freshness of values, ordered comparisons require reasoning modulo order-preserving renamings of data values. Such renamings are not unique, but we show that a canonical representative suffices to decide the relevant equivalence relations between words (cf. Lemma 12), making the learning independent of the choice of mapping. This breaks standard observation-table techniques based on suffix reasoning, which compare rows via equality of abstract future behavior.

To address this, table entries here are compared modulo canonical renamings and the subset of values stored in memory, yielding our notion *S-consistency* (cf. Section 4), which is not an equivalence relation and requires refined counterexample handling that updates symbolic transitions rather than only separating states. We implement this learning algorithm and demonstrate its effectiveness on a family of data languages and randomly generated DRAs.

We further study the complexity of decision problems for DRAs over ordered data domains, which are closely related to the queries arising in learning frameworks. We show PSPACE-completeness of language inclusion/intersection non-emptiness for DRAs over equality and ordered domains. For dense ordered domains, we additionally show PSPACE upper bounds for configuration equivalence, language equivalence, and memorability. For the memorability problem, our PSPACE upper bounds improve the previously known NEXPTIME upper bounds from [7, 25].

Related Work. Active learning of RAs has been investigated in [6, 9, 18, 23]. In [6], Permutation Deterministic Register Automata (PDRAs) were introduced, and our DRAs restricted to identity tests correspond to LAR-DRAs, a subclass of RAs with a fixed permutation policy. It was shown in [6] that minimization of LAR-DRAs is in PTIME, as both language equivalence and memorability are tractable. Moreover, they provide a PTIME active learning algorithm for LAR-DRAs using memorability queries. The learning algorithms in [18, 23] consider the DRA model of [11] and do not use memorability queries; their runtime is exponential in both the number of registers and the length of the longest counterexample. Additionally, there is work [28] on learning nominal automata, which are equivalent to RAs in expressiveness [8]. These automata group infinite states into orbits under permutations; specifically, [28] provides an active learning algorithm for a subclass of nondeterministic nominal automata—including

⁴ With only classical membership and equivalence queries, a learning algorithm would likely require exponential time, as discussed in Sect. 5

deterministic ones—over infinite domains with equality tests. The authors of [28] briefly note that their framework is intended to accommodate both equality and ordered domains, although working out the extension to ordered domains is left open. Venhoeck, Moerman and Rot [39] further study computation over ordered nominal sets, contributing techniques for reasoning about infinite ordered structures in nominal settings. A passive learning algorithm for LAR-DRA is given in [5].

Active learning for DRAs over ordered domains is studied in [10, 12], where data words consist of symbols from both a finite and an infinite set. Such data words are also considered in [23] and can be categorized into different patterns based on their finite components. This approach relies on tree queries instead of standard membership queries: for a given concrete prefix and a set S of abstract suffixes, the teacher returns a symbolic decision tree. This tree captures the behavior of the unknown automaton’s runs on the prefix followed by all possible concrete suffixes that satisfy the patterns in S . This tree-query technique is also used in [18] to reduce query complexity. Compared with memorability queries, tree queries provide richer information, since the resulting symbolic decision trees can expose both relevant register values and transition guards. In contrast, a memorability query directly returns the values of a prefix that may affect future behavior, independently of a particular suffix, and thus provides a more specialized oracle for identifying the required register contents. In [10, 12], ordered domains with a ternary operation $\text{sum}(\ell, m, n)$ (signifying $\ell + m = n$) and those with an operation $\text{inc}(m, n)$ (signifying $m + 1 = n$) are also studied. Furthermore, the equivalence and emptiness problems are addressed for the more expressive model in [14], which supports ordered data domains and allows certain linear arithmetic constraints.

On the other hand, our DRAs over ordered data domains were first introduced in [7, 25]. In this model, the duplication of register values is prohibited; this restriction does not reduce expressiveness and ensures the uniqueness of minimal automata. In our learning algorithm, in addition to equivalence and membership queries, the teacher also answers memorability queries. These queries were introduced in [6] to improve the complexity of learning DRAs over domains restricted to equality tests.

Symbolic finite automata (SFAs) constitute another automata model for infinite alphabets. Unlike RAs, SFAs retain the classical closure and decidability properties of finite automata but have strictly less expressive power [17]. There is extensive work on active learning for general and restricted classes of SFAs, see, for example, [3, 4, 19, 20, 22, 26, 27].

2 Preliminaries

We use $\mathbb{R}$ (resp. $\mathbb{Q}$, $\mathbb{Z}$, $\mathbb{N}$) to denote the sets of real (resp. rational, integer, and non-negative) numbers. A set $S \subseteq \mathbb{R}$ is *dense* in $\mathbb{R}$ if, for every $s < t$, there exists $r \in S$ with $s < r < t$ [35]. Accordingly, $\mathbb{Q}$ is dense in $\mathbb{R}$, whereas $\mathbb{Z}$ is not. For simplicity, we say $\mathbb{Q}$ and $\mathbb{R}$ are dense and $\mathbb{Z}$ is non-dense throughout the paper.

Let Σ be an alphabet and R a binary relation on Σ , where Σ may be finite or infinite. A *(data) word* is a finite sequence over Σ . For words $u = a_1 \dots a_m$, $v = b_1 \dots b_n$ in Σ^* and a symbol (or letter) $d \in \Sigma$, we write $d \in u$ if $d = a_i$ for some $1 \leq i \leq m$, and $u \cdot v$ denotes the concatenation $a_1 \dots a_m b_1 \dots b_n$. The length of u is denoted by $|u|$. The relation R induces an equivalence relation $\sim_R$ on Σ^* : for $u = a_1 \dots a_m$ and $v = b_1 \dots b_n$, we write $u \sim_R v$ if and only if (1) $m = n$, and (2) $(a_i, b_j) \in R$ if and only if $(b_i, b_j) \in R$ for all $1 \leq i, j \leq n$.

Each equivalence class τ of $\sim_R$ is called a $\sim_R$ -word type, or simply a *type*. We say τ has length n if every word u of type τ has $|u| = n$. A language $L \subseteq \Sigma^*$ is a *data language* over (Σ, R) if, for all words u, v of the same $\sim_R$ -type, $u \in L \Leftrightarrow v \in L$. In this paper, we consider only alphabet $\Sigma \in \{\mathbb{R}, \mathbb{Q}, \mathbb{Z}\}$ and R being either $<$ (the total order) or $=$ (the identity relation) on Σ .

Let σ be a mapping on Σ^* with $\sigma(u \cdot v) = \sigma(u) \cdot \sigma(v)$ for all $u, v \in \Sigma^*$. We call σ *R-preserving* if $\sigma(u) \sim_R u$ for all $u \in \Sigma^*$; when R is $<$, it is *order-preserving*. For any $u = a_1 \dots a_n, v = b_1 \dots b_n \in \Sigma^*$ with $u \sim_R v$, there exists an R -preserving partial mapping σ such that $\sigma(u) = v$. Moreover, if Σ is dense, σ can be chosen bijective as follows. For $(\Sigma, R) = (\mathbb{Q}, <)$, let $u' = a'_1 \dots a'_n$ and $v' = b'_1 \dots b'_n$ be the sorted sequences of u and v , respectively. Then $\sigma_{u \rightarrow v}$ is defined as follows: (1) $\sigma_{u \rightarrow v}(c) = \frac{(c - a'_i)(b'_{i+1} - b'_i)}{a'_{i+1} - a'_i} + b'_i$, for each $a'_i \neq a'_{i+1}$ and $a'_i \leq c < a'_{i+1}$; (2) $\sigma_{u \rightarrow v}(a'_i) = b'_i$, for each $a'_i = a'_{i+1}$, where $i = 1, \dots, n-1$; (3) $\sigma_{u \rightarrow v}(c) = (c - a'_1) + b'_1$, if $c < a'_1$; (4) $\sigma_{u \rightarrow v}(c) = (c - a'_n) + b'_n$, if $c \geq a'_n$. By definition, $\sigma_{u \rightarrow v}$ is order-preserving and bijective. Since $\sigma_{u \rightarrow v}$ is bijective, its inverse $\sigma_{u \rightarrow v}^{-1}$ also exists.

Register automata (RAs), also known as *finite-memory automata*, extend finite-state automata to infinite alphabets and recognize data languages. Several equivalent variants exist in the literature [6, 7, 24, 33]. In this paper, we adopt the definition of RAs from [7].

Definition 1. Given $k \in \mathbb{N}$, an alphabet Σ , and a binary relation R on Σ , a *k-register automaton* (*k-RA*) $\mathcal{A}$ over (Σ, R) is a tuple (Q, q_0, F, Δ) where: (1) Q is a set of states, partitioned into disjoint subsets $Q_0, \dots, Q_k$, (2) $q_0 \in Q_0$ is the initial state, (3) $F \subseteq Q$ is the set of final states, and (4) Δ is a finite set of transitions of the form (p, τ, E, q) , where $p \in Q_i$ and $q \in Q_j$ for some $0 \leq i, j \leq k$, τ is a $\sim_R$ -word type of length $i+1$, $E \subseteq \{1, \dots, i+1\}$, and $j = i+1 - |E|$.

An RA $\mathcal{A}$ is *deterministic* (DRA) if, for any two transitions (p, τ, E, q) and (p', τ', E', q') , we have $(E, q) = (E', q')$ whenever $(p, \tau) = (p', \tau')$.

A *configuration* of $\mathcal{A}$ is a pair (q, v) , where $q \in Q_i$ and v is a word over Σ of length i , for some $0 \leq i \leq k$. For configurations (p, u) and (q, v) and a symbol $a \in \Sigma$, we write $(p, u) \xrightarrow{a}_{\mathcal{A}} (q, v)$ or $(p, u) \xrightarrow{\tau: E}_{\mathcal{A}} (q, v)$ if there exists a transition $\delta = (p, \tau, E, q)$ such that $u \cdot a = a_1 \dots a_n$ is of type τ and v is obtained from $u \cdot a$ by removing all a_i with $i \in E$, subject to the following two conditions: (1) if $|u| = k$, then $E \neq \emptyset$ to prevent memory overflow; and (2) $j \in E$ whenever $1 \leq j < n$ and $a_j = a_n = a$, which ensures no data duplication by requiring that

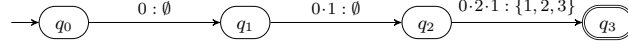


Fig. 1: A well-typed DRA $\mathcal{A}_{mid}$ recognizing L_{mid} where sink state and its related transitions are omitted.

if the current input a is already stored, it must be removed. The subscript $\mathcal{A}$ is omitted when clear from context.

Let $w = b_1 \dots b_m \in \Sigma^*$. A sequence of configurations $\pi = (p_0, u_0) \dots (p_m, u_m)$ is a *run* on w , written $(p_0, u_0) \xrightarrow{w} (p_m, u_m)$, if $(p_i, u_i) \xrightarrow{b_{i+1}} (p_{i+1}, u_{i+1})$ for all $0 \leq i < m$. It is *accepting* if $(p_0, u_0) = (q_0, \varepsilon)$ and $p_m \in F$. We say $\mathcal{A}$ is *complete* if every word $u \in \Sigma^*$ has a run in $\mathcal{A}$ from (q_0, ε) . The language recognized by $\mathcal{A}$ is $L(\mathcal{A}) = \{w \in \Sigma^* \mid (q_0, \varepsilon) \xrightarrow{w} (q_f, u), q_f \in F, u \in \Sigma^*\}$, which is a data language over (Σ, R) . Two RAs $\mathcal{A}$ and $\mathcal{A}'$ are *equivalent* if $L(\mathcal{A}) = L(\mathcal{A}')$.

Consider $L_{mid} = \{a_1 a_2 a_3 \mid a_1 < a_3 < a_2\}$ over $(\mathbb{Z}, <)$: the DRA in Fig. 1 has states $q_0, \dots, q_3$, with q_0 being the initial state and q_3 a final state, and recognizes L_{mid} . Each transition is labeled with $\tau : E$. For example, the transition from q_2 to q_3 is labeled with $0.2.1 : \{1, 2, 3\}$, where $0.2.1$ is the word type τ and $\{1, 2, 3\}$ is E . On the word $w = 3 \cdot 9 \cdot 7$, the DRA has the accepting run $\pi = (q_0, \varepsilon) \xrightarrow{3} (q_1, 3) \xrightarrow{9} (q_2, 3 \cdot 9) \xrightarrow{7} (q_3, \varepsilon)$, so $w \in L_{mid}$.

We denote by $|\mathcal{A}|$ the number of states in $\mathcal{A}$. An RA $\mathcal{A}$ is *state-minimal* if $|\mathcal{A}| \leq |\mathcal{A}'|$ for every equivalent RA $\mathcal{A}'$, and *data-minimal* if it is a k -RA and every equivalent $\mathcal{A}'$ is a k' -RA with $k' \geq k$. $\mathcal{A}$ is *minimal* if it is both state- and data-minimal. A state-minimal or data-minimal RA recognizing $L(\mathcal{A})$ always exists, but a minimal RA (even a DRA) need not. Consider the following example.

Example 2. For each $n \in \mathbb{N}$, let L_n be the language of words w over $\mathbb{Q}$ that form a strictly increasing or decreasing sequence of length n . For example, when $n = 3$, the word $w = 2 \cdot -1 \cdot -3.5$ belongs to L_n , whereas $u = 1 \cdot 2$ and $v = 2 \cdot -3 \cdot 0$ do not. Consider a 1-DRA $\mathcal{A}_n$ that recognizes L_n . It has two fashions—increasing and decreasing—and always stores the most recent input in its single register. Given $w = a_1 \dots a_m$, $\mathcal{A}_n$ enters the increasing (resp. decreasing) fashion if $a_2 > a_1$ (resp. $a_2 < a_1$). For $2 < i < n$, it remains in the current fashion provided a_i is larger (resp. smaller) than the value stored in the register; otherwise it rejects. Length $m = n$ is verified using control states. Since both fashions require $n - 1$ intermediate states (plus initial and final states), $\mathcal{A}_n$ has $2n$ states.

While $\mathcal{A}_n$ is data-minimal for L_n , it is not state-minimal since an equivalent $(n-1)$ -DRA $\mathcal{A}'_n$ with $|\mathcal{A}'_n| < |\mathcal{A}_n|$ exists. This DRA stores the first $n-1$ symbols of the input in its registers and checks on the fly whether these values form a strictly increasing or strictly decreasing sequence. It needs $n+2$ states: $n+1$ states to verify the word length and one rejecting state to make $\mathcal{A}'_n$ complete. $\square$

As shown in Example 2, a minimal DRA need not exist for every data language L . To address this, we introduce *well-typeness*, which ensures the existence of a *minimal well-typed* DRA for every DRA-recognizable language. Notably, the

canonical DRAs of [7] also satisfy this condition. Restricting DRAs to be well-typed preserves expressive power, though it may increase the number of states.

Definition 3. Let $\mathcal{A}$ be an RA over (Σ, R) . We call $\mathcal{A}$ well-typed if for every two configuration transitions $(p, u) \xrightarrow{a}_{\mathcal{A}} (q, v)$ and $(p', u') \xrightarrow{a'}_{\mathcal{A}} (q, v')$ ending in the same state q , we have $v \sim_R v'$.

Intuitively, an RA is well-typed if the $\sim_R$ -type of its registers is uniquely determined by the current state. In the remainder of the paper, *all DRAs* are assumed to be *well-typed* unless stated otherwise.

3 DRAs over Non-Dense Ordered Domains

In this section, we will present the Myhill–Nerode theorem for DRAs over ordered domains, which forms the basis for learning and minimization. We then solve open problems for DRAs over *non-dense* domains—most notably minimization and configuration reachability. Together, these results lay the foundation for a unified learning framework for DRAs over both dense and non-dense settings.

As with other state machines, minimization for DRAs is tightly related to the Myhill–Nerode theorem, as shown in [7]. We now introduce some notions needed to state this theorem for DRAs. For a language L and two words u, v , a word w is called an *L -distinguishing extension* for u and v if exactly one of the words $u \cdot w$ and $v \cdot w$ belongs to L . For regular languages L , two words are Nerode-congruent when no L -distinguishing extension exists. For data languages, however, word classification additionally requires the notion of *memorability* introduced in [7]. Consider the language L_7 in Example 2. Any DRA $\mathcal{A}$ recognizing L_7 , after reading $u = 2 \cdot 3 \cdot 6$, must store the last value 6 in a register, since $\mathcal{A}$ must check whether the next input value is greater than 6. Thus, 6 is *L -memorable* in u , whereas 2 and 3 are not. The formal definition is as follows:

Definition 4. Let L be a language over (Σ, R) . A symbol $a \in \Sigma$ is *L -memorable* in a word $u \in \Sigma^*$ if there exist a word u' of the same $\sim_R$ -type as u , symbols $a', b' \in \Sigma$, a word $w \in \Sigma^*$ and an R -preserving mapping σ with $\sigma(a') = b'$ and $\sigma(d) = d$ for all other $d \in u' \cdot w$, such that: (1) $u \cdot a \sim_R u' \cdot a' \sim_R \sigma(u' \cdot a')$; and (2) w is an *L -distinguishing extension* for u' and $\sigma(u')$.

Intuitively, only memorable symbols determine the membership of future extended words. If Σ is dense, we can let $u' = u$, and replace a in u with a nearby symbol b , obtaining a word $\sigma(u)$ with $\sigma(u) \sim_R u$ and check whether this change affects the membership of its extensions in L , i.e., there is an *L -distinguishing extension* w for u and $\sigma(u)$. In the non-dense setting, such a replacement b may not exist. We therefore need to construct a word u' of the same type whose data values have sufficient space to allow a valid replacement b' and then test on u' .

We write $\text{mem}_L(u) = a_1 \dots a_k$ for the sequence of *L -memorable* symbols in u , ordered so that for every a_i, a_j , $j > i$ if the last occurrence of a_j in u occurs after the last occurrence of a_i . Let $\mathcal{A}$ be a DRA recognizing L . If $(q_0, \varepsilon) \xrightarrow{u}_{\mathcal{A}} (q, v)$

for some configuration (q, v) , then by definition, $\text{mem}_L(u)$ is a subsequence of v [7]. We are now ready to define the Nerode congruence for data languages.

Definition 5. (cf. [7]) *Given a data language L over (Σ, R) , we define an equivalence relation $\cong_L$ on Σ^* , where for every $u, v \in \Sigma^*$, we write $u \cong_L v$ if:*

- $\text{mem}_L(u) \sim_R \text{mem}_L(v)$, and
- for all $x, y \in \Sigma^*$, if $\text{mem}_L(u) \cdot x \sim_R \text{mem}_L(v) \cdot y$, then $u \cdot x \in L \Leftrightarrow v \cdot y \in L$.

Intuitively, equivalent words u and v must have memorable words of the same type. Furthermore, when their memorable-word extensions share the same type, extending u and v by the corresponding suffixes yields the same membership in L , since only memorable words affect future membership.

Let L be a data language over (Σ, R) and $k = \max\{|\text{mem}_L(u)| \mid u \in \Sigma^*\}$. Let $[u]_{\cong_L}$ denote the equivalence class defined by $\cong_L$ where $u \in \Sigma^*$ belongs to. We define the *canonical k -RA* $\mathcal{C}_L = (Q, q_0, F, \Delta)$ for L as follows:

- $Q = \bigcup_{i=0}^k Q_i$ where $Q_i = \{[u]_{\cong_L} \subseteq \Sigma^* \mid u \in \Sigma^*, |\text{mem}_L(u)| = i\}$,
- $q_0 = [\varepsilon]_{\cong_L}$, $F = \{[u]_{\cong_L} \in Q \mid u \in L\}$, and
- $([u]_{\cong_L}, \tau, E, [u']_{\cong_L}) \in \Delta$ if there exists a symbol $a \in \Sigma$ such that:
 - $u \cdot a \cong_L u'$, $\text{mem}_L(u) \cdot a$ has type τ , and
 - E is the set of indices i in the sequence $a_1 \dots a_d = \text{mem}_L(u) \cdot a$ such that either $a_i \notin \text{mem}_L(u \cdot a)$ or $a_i = a$ for $i < d$.

In general, $\mathcal{C}_L$ may have infinitely many states or registers and may be non-deterministic. When L is DRA-recognizable, $\mathcal{C}_L$ is exactly the minimal DRA for L . This Myhill–Nerode theorem for DRAs was established in [7]:

Theorem 6. (cf. [7]) *Let L be a data language. Then L is recognized by a DRA iff $\cong_L$ has finite index. Moreover, every DRA-recognizable language L has a canonical automaton $\mathcal{C}_L$ which is the unique minimal DRA up to isomorphism.*

With Theorem 6, the standard DFA minimization algorithm extends to DRAs. Given a DRA $\mathcal{A}$, we merge states p and q whenever the residual languages of the configurations (p, u) and (q, v) , for some u, v , coincide. The existence of such u and v also ensures that p and q are reachable from the initial configuration, with u and v serving as witnesses. Checking this language equivalence reduces to solving the *configuration reachability problem* in the product automaton. Consequently, the minimization algorithm⁵ depends on solving the **configuration reachability problem**, which is interesting in its own right: Given a configuration (p, u) and a state q , determine whether there exists a word w_q such that a run from (p, u) reaches some configuration (q, w) on input word w_q .

As shown in [7], the configuration reachability problem is trivially decidable for dense domains. Given a configuration (p, u) and a transition (p, τ, E, q) , the density property ensures that there always exists a value a such that $u \cdot a$ has

⁵ Complete reductions and pseudocode for minimizing DRAs are provided in App. A.2.

type τ . Hence, with a complete DRA, checking whether a configuration (p, u) can reach a state q reduces to deciding whether there exists a sequence of transitions from (p, u) that leads to the state q . This can be checked easily.

The decidability of the configuration reachability problem in the non-dense setting, however, remains *open* since the above approach fails. Consider the DRA $\mathcal{A}_{mid}$ over $(\mathbb{Z}, <)$ in Fig. 1: we have $(q_2, u) \xrightarrow{a} (q_3, \varepsilon)$ when $u = 1 \cdot 5$ and $a = 3$, but for $u = 1 \cdot 2$, no value of a produces such a run. To address this gap, we provide the *first* decidable complexity result of this problem over non-dense domains.

Theorem 7. *The configuration reachability problem for DRAs over $(\mathbb{Z}, <)$ is decidable.*

As an immediate result of Theorem 7, we obtain the following:

Corollary 8. *The minimization problem for DRAs over $(\mathbb{Z}, <)$ is decidable.*

In fact, a simpler route to achieve Corollary 8 is available. The next theorem shows that minimization and language equivalence for DRAs over both dense and non-dense domains fit into a single unified framework. That is, we can reduce these problems over non-dense domains to the problems over dense domains which are already known to be decidable [7]. However, this unification does *not* extend to the configuration reachability problem.

Theorem 9. *Let $\mathcal{A}$ be a DRA, and let $L_{\mathbb{Q}}$ and $L_{\mathbb{Z}}$ be the data languages recognized by $\mathcal{A}$ over $(\mathbb{Q}, <)$ and $(\mathbb{Z}, <)$, respectively. For any $w \in \mathbb{Q}^*$ and $w' \in \mathbb{Z}^*$ with $w \sim_{<} w'$, it holds that $w \in L_{\mathbb{Q}}$ iff $w' \in L_{\mathbb{Z}}$. Furthermore, a DRA over $(\mathbb{Q}, <)$ is the minimal DRA recognizing $L_{\mathbb{Q}}$ iff the same DRA over $(\mathbb{Z}, <)$ is the minimal DRA recognizing $L_{\mathbb{Z}}$. This correspondence remains valid when either $\mathbb{Q}$ or $\mathbb{Z}$ is replaced by $\mathbb{R}$.*

Proof. Given a DRA $\mathcal{A}$, we use $\mathcal{A}_{\mathbb{Q}}$ and $\mathcal{A}_{\mathbb{Z}}$ to represent the DRAs $\mathcal{A}$ over $(\mathbb{Q}, <)$ and $(\mathbb{Z}, <)$, respectively. For every $w \in \mathbb{Q}^*$, there exists $w' \in \mathbb{Z}^*$ s.t. $w' \sim_{<} w$, and vice versa. Therefore, $L(\mathcal{A}_{\mathbb{Z}}) = L(\mathcal{A}_{\mathbb{Q}}) \cap \mathbb{Z}^*$ and $L(\mathcal{A}_{\mathbb{Q}}) = \{w \mid \exists w' \in L(\mathcal{A}_{\mathbb{Z}}) \wedge w' \sim_{<} w\}$. This implies that $\mathcal{A}_{\mathbb{Q}}$ is the minimal DRA recognizing $L(\mathcal{A}_{\mathbb{Q}})$ if and only if $\mathcal{A}_{\mathbb{Z}}$ is the the minimal DRA recognizing $L(\mathcal{A}_{\mathbb{Z}})$. The argument remains valid if the ordered domain $\mathbb{Q}$ or $\mathbb{Z}$ is replaced by $\mathbb{R}$. $\square$

4 Active Learning of DRAs

In this section, we present an algorithm for learning a DRA of an unknown data language L using *polynomially* many queries in the active learning framework. In this framework, a *learner* aims to infer an automaton recognizing L , while a *teacher* knows L and answers *membership* queries $\text{MQ}(u)$ (checking whether $u \in L$) and *equivalence* queries $\text{EQ}(\mathcal{H})$ (checking whether $\mathcal{H}$ recognizes L).

The learner begins by issuing membership queries to populate an *observation table* $\mathcal{T}$, from which it constructs a conjectured automaton $\mathcal{H}$. An equivalence query then checks its correctness: if the teacher replies “Yes”, learning terminates;

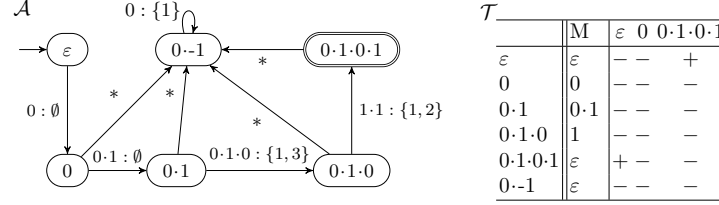


Fig. 2: The example DRA $\mathcal{A}$ where the missing transitions are labeled with $*$, and the observation table $\mathcal{T}$ where the extension set X are omitted.

otherwise a counterexample $w \in L \ominus L(\mathcal{H})$ is returned and used to refine $\mathcal{H}$. This process repeats until a correct automaton is found. While originally developed for DFAs, we adapt this framework to DRAs over ordered domains. By Theorem 9, it suffices to learn DRAs over $(\mathbb{Q}, <)$, since the case of $(\mathbb{Z}, <)$ reduces to it; See Section C for more details on the reduction.

Unlike DFA learning, data languages lack known polynomial-time learning algorithms with respect to the number of queries, even for only equality tests on data [23]. This barrier can be overcome by using also *memorability queries* [6], $\text{Mem}(u)$, which return the memorable word $\text{mem}_L(u)$ of a word u .

We first present a DRA learner using membership, equivalence, and memorability queries, and then give improved complexity results for some decision problems of DRAs that are closely related to solving queries.

4.1 Learning DRAs with Memorability Queries

We fix a target DRA-recognizable language L over $(\Sigma, R) = (\mathbb{Q}, <)$ in the whole section. The learner is assumed to know (Σ, R) and maintains an observation table to record its knowledge of L .

Definition 10 (Observation Table). An observation table $\mathcal{T}$ for a DRA over (Σ, R) is a tuple (U, X, S, M, f) , where U , X , and S are finite sets of data words: U is the set of prefixes, X the extensions, and S the suffixes. The function $M : U \cup X \rightarrow \Sigma^*$ records the memorable word of each word $u \in U \cup X$, i.e., $M(u) = \text{mem}_L(u)$. The classification function $f : U \times S \rightarrow \{+, -\}$ labels each $u \cdot w$ as in L or not, i.e., $f(u, w) = \text{MQ}(u \cdot w)$. We require (1) $U \cup X$ to be prefix-closed and (2) $\epsilon \in U$ and $\epsilon \in S$.

Table $\mathcal{T}$ in Fig. 2 shows a final observation table for $L_{\text{rep}} = \{xyxy \in \mathbb{Q}^4 \mid x < y\}$. The rows correspond to the prefix set $U = \{\epsilon, 0, 0-1, 0-1-0, 0-1-0-1, 0-1-1\}$, and the columns contain the memorable function M and the suffix set $S = \{\epsilon, 0, 0-1-0-1\}$. The set X is omitted here, as it is only used internally when constructing conjectured DRAs. From this table, we can obtain the minimal DRA $\mathcal{A}$ (with respect to $\cong_L$), also shown in Fig. 2. Each row $u \in U$ naturally corresponds to a state of $\mathcal{A}$: u serves as the representative of its $\cong_L$ -equivalence

class, and $\mathcal{A}$ reaches the associated state exactly after reading u . For instance, the empty word ε is the initial state, whereas $0 \cdot 1 \cdot 0 \cdot 1$ is the only final state.

In the remainder of the paper, we abusively use the word representatives in U as state names. While constructing the observation table, as in the classic L^* algorithm for DFAs, we ensure that for every state $u \in U$, each one-letter extension $u \cdot b$ appears in the table, i.e., in X . To compute the one-letter extensions of u (i.e., $u \cdot b$ for $b \in \Sigma$), we observe that (i) only the memorable word $M(u)$ is relevant, since the types of future words depend solely on its memorable letters; and (ii) we do not need to enumerate all $b \in \Sigma$, but only a finite set that covers all possible types of $M(u) \cdot b$.

Let $a_1 < \dots < a_d$ be the ordered sequence of *distinct* letters in $M(u)$. Then b can take one of the four types of values:

1. $b \in M(u)$, i.e., it means that b occurs also in $M(u)$;
2. $b = a_1 - 1$, i.e., we have that $b < a_i$ for all $1 \leq i \leq d$;
3. $b = a_d + 1$, i.e., it gives that $b > a_i$ for all $1 \leq i \leq d$; and finally
4. $b = (a_i + a_{i+1})/2$ for some $1 \leq i < d$, i.e., $a_j < b < a_k$ for all $j \leq i$ and $k \geq i + 1$.

Note that the fourth type of values will be ignored if $|M(u)| = 1$ and we set $b = 0$ when $u = \varepsilon$. If R is the identity test, it suffices to add values $b \in M(u)$ or a fresh letter $b \notin M(u)$. For example, if $M(u) = 0 \cdot 1$, then $b \in \{0, 1, 2\}$.

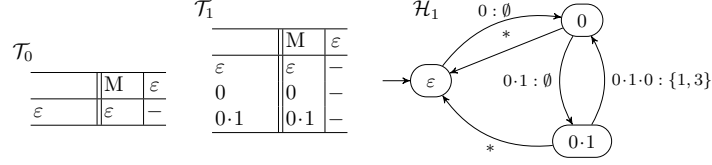
To ensure that U matches the state set of the target DRA, we must determine when two words $u_1, u_2 \in U$ are *nonequivalent* under the Nerode congruence $\cong_L$ (Definition 5). Recall that $u_1 \not\cong_L u_2$ holds precisely when (i) their memorable word types differ, i.e., $\text{mem}_L(u_1) \not\sim_R \text{mem}_L(u_2)$, or (ii) there exist $x, y \in \Sigma^*$ such that $\text{mem}_L(u_1) \cdot x \sim_R \text{mem}_L(u_2) \cdot y$ but $u_1 \cdot x \in L \not\equiv u_2 \cdot y \in L$. While testing whether two words have the same type is straightforward, finding witness words x and y is more challenging. In contrast, the classical L^* algorithm for DFAs [2] needs only a *single* witness to distinguish two words.

Fortunately, since Σ is dense, it suffices to find an order-preserving mapping σ with $\sigma(\text{mem}_L(u_1)) = \text{mem}_L(u_2)$ and a *single* L -distinguishing suffix w that separates $\sigma(u_1)$ from u_2 . The following lemma justifies this reduction:

Lemma 11. *Let Σ be a dense set. For two words $u_1, u_2 \in \Sigma^*$ with $\text{mem}_L(u_1) \sim_R \text{mem}_L(u_2)$, $u_1 \not\cong_L u_2$ if and only if there exist a bijective order-preserving mapping σ and a word w such that $\sigma(\text{mem}_L(u_1)) = \text{mem}_L(u_2)$ and $\sigma(u_1) \cdot w \in L \not\equiv u_2 \cdot w \in L$.*

A further challenge is that infinitely many order-preserving mappings σ may exist, making enumeration impossible. Nevertheless, we prove that it suffices to find *one* suitable mapping. Notably, such a bijective order-preserving mapping σ with $\sigma(\text{mem}_L(u_1)) = \text{mem}_L(u_2)$ can be obtained easily, as described in Sect. 2.

Lemma 12. *Let Σ be a dense set, and let u_1, u_2 be distinct words. If σ is a bijective order-preserving mapping with $\sigma(\text{mem}_L(u_1)) = \text{mem}_L(u_2)$, then $u_1 \not\cong_L u_2$ iff there exists $w \in \Sigma^*$ such that $\sigma(u_1) \cdot w \in L \not\equiv u_2 \cdot w \in L$.*

Fig. 3: Unclosed table $\mathcal{T}_0$, closed table $\mathcal{T}_1$ and its induced DRAs $\mathcal{H}_1$.

To derive a conjectured DRA from the observation table $\mathcal{T}$, the table must be *closed*. We introduce the notion of *S-consistency*: a word $v \in X$ is said to be *S-consistent* with a prefix $u \in U$ if $M(v) \sim_R M(u)$ and, for every $w \in S$, we have $\text{MQ}(\sigma(v) \cdot w) = f(u, w)$, where σ is an order-preserving mapping on Σ s.t. $\sigma(M(v)) = M(u)$. By Lemma 12, the table $\mathcal{T}$ is said to be closed if, for each extension $ua \in X$ of a word $u \in U$, there exists a prefix $u' \in U$ s.t. ua is *S-consistent* to u' . Intuitively, the target DRA $\mathcal{A}$ moves to state u' at state u when reading $a \in \Sigma$ if ua is *S-consistent* with u' . Note that *S-consistency* is not an equivalence relation, as it does not need to be symmetric nor transitive.

Our algorithm relies on two key properties of *S-consistency*: (i) if u is not *S-consistent* with v , then $u \not\approx_L v$; (ii) if u is not *S-consistent* with v , then u is not *S'*-consistent with v whenever $S \subseteq S'$.

If $\mathcal{T}$ is not closed, the learner invokes `CloseTable( $\mathcal{T}$ , MQ, Mem)`, which repeatedly: (i) finds a word $ua \in X$ with $u \in U$ that has no *S-consistent* $u' \in U$; (ii) adds ua to U and adds also all its one-letter extensions uab to X ; (iii) fills the new row labeled with ua by setting $f(ua, w) = \text{MQ}(ua \cdot w)$ for all $w \in S$ and updates $M(uab) = \text{Mem}(uab)$ for all new extensions $uab \in X$.

Consider the initial table $\mathcal{T}_0$ in Fig. 3 for learning $L = \{xyxy \in \mathbb{Q}^4 \mid x < y\}$ with $X = \{0\}$ and $M(0) = 0$. To close the table, the learner checks whether $\varepsilon \cdot 0$ is *S-consistent* with ε . Since $M(\varepsilon \cdot 0) = 0$ differs in word type from $M(\varepsilon) = \varepsilon$, the check fails, and 0 is added to U . We update f with $f(0, \varepsilon) = \text{MQ}(0 \cdot \varepsilon) = -$ for the new row 0 and column ε . Further, the one-letter extensions of 0 are $\{0 \cdot 0, 0 \cdot -1, 0 \cdot 1\}$, so X becomes $\{0, 0 \cdot 0, 0 \cdot -1, 0 \cdot 1\}$. Using memorability queries, we also update M as $M(0 \cdot 0) = \varepsilon$, $M(0 \cdot -1) = \varepsilon$, $M(0 \cdot 1) = 0 \cdot 1$. Continuing in this way, the learner adds 0·1 to U and obtains the closed table $\mathcal{T}_1$ in Fig. 3.

With all extensions computed for each new state, the conjectured DRA $\mathcal{H}$ we construct will be complete. Once $\mathcal{T}$ is closed, we can induce $\mathcal{H}$ using Definition 13.

Definition 13 (DRA Construction). Let $\mathcal{T} = (U, X, S, M, f)$ be a closed observation table and $k = \max\{|M(u)| \mid u \in U\}$. We construct a k -DRA $\mathcal{H} = (Q, q_0, F, \Delta)$ from $\mathcal{T}$ as follows:

- $Q = \bigcup_{i=0}^k Q_i$ where $Q_i = \{u \in U \mid |\text{Mem}(u)| = i\}$, $q_0 = \varepsilon \in Q_0$, $F = \{u \in U \mid f(u, \varepsilon) = +\}$, and
- $(u, \tau, E, u') \in \Delta$ if there is an extension $u \cdot a \in X$ s.t. (1) $u' \in U$ is the first found word with which $u \cdot a$ is *S-consistent*, (2) $\tau = M(u) \cdot a = a_1 \cdots a_d$, and E contains indices i where $a_i \notin M(u \cdot a)$ or $a_i = a_d = a$ with $i < d$.

Algorithm 1: Function $\text{cexAnalyze}(\mathcal{T}, \mathcal{H}, w, \text{MQ}(\cdot))$ that analyses w and returns a suffix v . The prerequisite is that $w \in L \ominus L(\mathcal{H})$.

```

1  $u \leftarrow \varepsilon, r \leftarrow \varepsilon, mq \leftarrow \text{MQ}(w);$ 
2 while True do
3   Let  $a \cdot z := w$  where  $a \in \Sigma$ ; //  $w = \varepsilon$  is impossible;
4   //INVARIANT:  $r = M(u)$ ;
5   Let  $(u, r) \xrightarrow{M(u) \cdot b : E} (u', r')$  be the transition taken in  $\mathcal{H}$  when reading  $a$ ;
6   //INVARIANT:  $r \cdot a = M(u) \cdot a \sim_R M(u) \cdot b$ ,  $M(u') \sim_R M(ub) = r'$ ;
7   Let  $\sigma_1$  be an order-preserving mapping from  $M(u) \cdot a$  to  $M(u) \cdot b$ ;
8   //Since  $r = M(u)$ ,  $r' = M(u \cdot b)$  is obtained by removing letters
   in positions  $E$  from  $r \cdot b$ ;
9   Let  $\sigma_2$  be an order-preserving mapping from  $M(u \cdot b)$  to  $M(u')$ ;
10  if  $\text{MQ}(u' \cdot \sigma_2(\sigma_1(z))) \neq mq$  then
11    | return new suffix  $\sigma_2(\sigma_1(z))$ ;
12  end
13   $u \leftarrow u', r \leftarrow M(u'), w \leftarrow \sigma_2(\sigma_1(z));$  //  $\text{MQ}(u' \cdot \sigma_2(\sigma_1(z))) = mq$ ;
14 end

```

It immediately follows from Theorem 6 that:

Lemma 14. *If U contains exactly the set of word representatives defined by $\cong_L$, then $\mathcal{H}$ is a minimal DRA recognizing the target language L .*

Consider constructing the DRA $\mathcal{H}_1$ from the closed table $\mathcal{T}_1$ in Fig. 3. Since the maximal length of a memorable word is $|0 \cdot 1| = 2$, $\mathcal{H}_1$ is a 2-DRA. The set of states is $Q = \{\varepsilon\} \uplus \{0\} \uplus \{0 \cdot 1\}$, with $q_0 = \varepsilon$, and $F = \emptyset$ because no $u \in U$ satisfies $f(u, \varepsilon) = +$. Here, we focus on the transition from 0 to $0 \cdot 1$ in Δ . The one-letter extension $0 \cdot 1$ of state 0 is S -consistent with $0 \cdot 1$, so reading letter 1 at 0 leads to state $0 \cdot 1$. We then set $\tau = M(0) \cdot 1 = 0 \cdot 1$ and $E = \emptyset$, since both 0 and 1 belong to $M(0 \cdot 1)$ and no letters need to be removed.

Once the conjectured DRA $\mathcal{H}$ is constructed, the learner poses an equivalence query $\text{EQ}(\mathcal{H})$ to the teacher. If the answer is positive, $\mathcal{H}$ is returned as the result. Otherwise, the teacher provides a counterexample $w \in L(\mathcal{H}) \ominus L$. The learner analyzes w to update the observation table, allowing it to correct $\mathcal{H}$ by adding new states or redirecting transitions. The counterexample analysis procedure is described in Algorithm 1. An example of counterexample analysis can be found in Section B.

In Algorithm 1, the counterexample w is processed letter by letter. In each iteration, we assume the current configuration is (u, r) and $\text{MQ}(u \cdot az) = mq$, where $w = a \cdot z$. Reading letter a at state u , $\mathcal{H}$ transitions to u' . However, this transition may have been constructed using a different letter b such that ub is S -consistent with u' and $a \neq b$, where $M(u) \cdot a \sim_R M(u) \cdot b$.

Our goal is to identify an extension ub wrongly classified as equivalent to u' . We first normalize the remaining suffix z for ub via the mapping σ_1 from $M(u) \cdot a$ to $M(u) \cdot b$, preserving membership results: $\text{MQ}(ub \cdot \sigma_1(z)) = mq = \text{MQ}(ua \cdot z)$,

Algorithm 2: The DRA learner

```

1 Initialize table  $\mathcal{T} = (U, X, S, M, f)$  where  $U = S = \{\varepsilon\}$ ,  $X = \{0\}$ ,  $M(\varepsilon) = \varepsilon$ ,
   $M(0) = \text{Mem}(0)$ , and  $f(\varepsilon, \varepsilon) = \text{MQ}(\varepsilon)$ ;
2  $\text{CloseTable}(\mathcal{T}, \text{MQ}(\cdot), \text{Mem}(\cdot))$  and let  $\mathcal{H}$  be the DRA constructed from  $\mathcal{T}$ ;
3 Let  $(res, w)$  be the response from the teacher on  $\text{EQ}(\mathcal{H})$ ;
4 while  $res = \text{No}$  do
5    $v \leftarrow \text{cexAnalyze}(\mathcal{T}, \mathcal{H}, w, \text{MQ}(\cdot))$ ;
6   Add  $v$  to the suffix set  $S$  in  $\mathcal{T}$  and set  $f(u, v) = \text{MQ}(u \cdot v)$  for all  $u \in U$ ;
7    $\text{CloseTable}(\mathcal{T}, \text{MQ}(\cdot), \text{Mem}(\cdot))$  and let  $\mathcal{H}$  be the DRA constructed from  $\mathcal{T}$ ;
8   Let  $(res, w)$  be the response from the teacher on  $\text{EQ}(\mathcal{H})$ ;
9 end
10 return  $\mathcal{H}$ ;

```

since both runs visit the same state sequence in the target DRA of L . To verify whether $ub \cong_L u'$, we further normalize $\sigma_1(z)$ for u' using σ_2 from $M(ub)$ to $M(u')$, yielding $v = \sigma_2(\sigma_1(z))$. If $\text{MQ}(u' \cdot v) \neq mq$, then $u' \not\cong_L ub$, according to Lemma 12 since $\sigma_2(ub \cdot \sigma_1(z)) \sim_R \sigma_2(ub) \cdot v$, giving $\text{MQ}(\sigma_2(ub) \cdot v) = mq$. Otherwise, we proceed to the next iteration with $u \leftarrow u'$, $r \leftarrow M(u')$, and $w \leftarrow v$. Since $w \in L \ominus L(\mathcal{H})$, before reaching the last state u'' where $\text{MQ}(u'' \cdot \varepsilon) \neq mq$, the membership results must change at some point. Hence, the **if** condition will eventually be satisfied, and a suffix v will be returned.

The correctness of Algorithm 1 is guaranteed by the following lemma.

Lemma 15. *Let w be the counterexample provided by the teacher. Algorithm 1 terminates after at most $|w|$ iterations and returns a suffix v such that $ub \in X$ is not $S \uplus \{v\}$ -consistent with a word $u' \in U$ with which ub is currently S -consistent.*

We now formally define the learner in Algorithm 2. To initialize the observation table $\mathcal{T}$, the learner inserts the empty word ε into U . The extension set is initialized as $X = \{0\}$, since 0 has the same type as all one-letter words. It fills entries via membership queries $\text{MQ}(\cdot)$ and memorability queries $\text{Mem}(\cdot)$, so $M(\varepsilon) = \varepsilon$ and $M(0) = \text{Mem}(0)$. The function CloseTable closes $\mathcal{T}$ with the help of $\text{MQ}(\cdot)$ and $\text{Mem}(\cdot)$. Conjectured DRAs $\mathcal{H}$ are then derived from $\mathcal{T}$ following Definition 13. On receiving a counterexample w , cexAnalyze (Algorithm 1) returns a suffix v that is added to S . New entries are filled via $\text{MQ}(\cdot)$. The refinement loop of $\mathcal{H}$ terminates once the teacher returns a positive answer.

The soundness and completeness of Algorithm 2 directly follow from Lemmas 12, 14 and 15.

Theorem 16. *Let $\mathcal{A}$ be a complete canonical minimal DRA of L with n states and m transitions defined under $\cong_L$, and let d be the maximum length of any counterexample returned by the teacher. It holds that:*

- Algorithm 2 terminates and returns a DRA $\mathcal{H}$ that is isomorphic to $\mathcal{A}$.
- Algorithm 2 terminates after at most $m \times (n - 1)$ equivalence queries.
- Algorithm 2 requires m memorability queries and $\mathcal{O}(d \times m \times n + m^3 \times n^3)$ membership queries.

The returned DRA $\mathcal{H}$ recognizes L , as confirmed by the teacher. Algorithm 2 terminates because each counterexample either corrects a transition or adds a new state (Lemma 15), and the number of equivalence classes under $\cong_L$ and transitions in target DRA $\mathcal{A}$ is finite. Now we show that our algorithm runs in polynomial time in the number of queries.

In DFA learning, two words u_1 and u_2 are considered equivalent with respect to a set of suffixes S if $\text{MQ}(u_1w) = \text{MQ}(u_2w)$ for all $w \in S$. Thus, when a transition ub is misclassified as consistent with some u' , it can safely be added to U . In our setting, however, S -consistency does not guarantee this property, as it is not an equivalence relation. When Algorithm 1 returns a distinguishing suffix v separating ub from the state u' with which ub is currently S -consistent, we add v to the suffix set S to correct the transition from u over b . Consequently, ub becomes not $S \uplus \{v\}$ -consistent with u' . Each transition can be corrected at most $n - 1$ times; that is, the target state of a transition over b from u may be misclassified no more than $n - 1$ times, as once ub is inconsistent with a state u'' , it remains inconsistent with u'' thereafter. Hence, the learner asks at most $m \cdot (n - 1)$ equivalence queries in total. In practice, new states are typically discovered immediately after adding the suffix returned by the counterexample analysis. The reasoning for membership and memorability queries can be found in Section D.

4.2 Complexity of Decision Problems

This section studies the complexity of some key decision problems for DRAs over ordered data domains. Understanding these complexities is essential, as they closely correspond to the queries required by our active learning algorithm, particularly when the teacher has the target data language in the form of a DRA. All results in this section apply to *both* well-typed and non-well-typed DRAs.

Let $\mathcal{A}_1 = (Q^1, q_0^1, F^1, \Delta^1)$ be a k_1 -DRA and $\mathcal{A}_2 = (Q^2, q_0^2, F^2, \Delta^2)$ be a k_2 -DRA, respectively. The *language intersection non-emptiness* asks whether there is a word $w \in L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$. The *language inclusion* problem asks whether $L(\mathcal{A}_1) \subseteq L(\mathcal{A}_2)$, equivalently whether $L(\mathcal{A}_1) \cap \overline{L(\mathcal{A}_2)} = \emptyset$, where $\overline{L(\mathcal{A}_2)}$ is obtained by complementing $\mathcal{A}_2$ via swapping accepting and non-accepting states since $\mathcal{A}_2$ is deterministic.

Theorem 17. *For DRAs over $(\Sigma, =)$ and over $(\Sigma, <)$, the language intersection non-emptiness problem is PSPACE-complete. Consequently, the language inclusion problem is also PSPACE-complete.*

For the PSPACE upper bound, intersection non-emptiness is reduced to reachability in the product of the two DRAs, where it suffices to distinguish register valuations by their *relative* data types. If an accepting run exists, there is one of length at most $|Q^1| \cdot |Q^2| \cdot (k_1 + k_2)!$, so it can be guessed symbol by symbol while storing only the current product configuration and a counter. PSPACE-hardness already holds for DRAs with identity tests, via a reduction from linear-bounded deterministic Turing machines.

Note that by Theorem 9, the above result holds for both dense and non-dense domains.

Given a DRA $\mathcal{A}$ over (Σ, R) and a configuration (q, v) of $\mathcal{A}$, we define the language of $\mathcal{A}^{(q, v)}$ as $L(\mathcal{A}^{(q, v)}) = \{w \in \Sigma^* \mid \exists q_f \in F, u \in \Sigma^*. (q, v) \xrightarrow{w}_{\mathcal{A}} (q_f, u)\}$. Let (q_1, v_1) and (q_2, v_2) be two configurations of $\mathcal{A}_1$ and $\mathcal{A}_2$ respectively. The *configuration equivalence problem* asks whether $L(\mathcal{A}_1^{(q_1, v_1)}) = L(\mathcal{A}_2^{(q_2, v_2)})$. The *language equivalence problem* asks whether $L(\mathcal{A}_1) = L(\mathcal{A}_2)$. It is a special case of the configuration equivalence problem, that is, it is equivalent to the configuration equivalence problem whether $L(\mathcal{A}_1^{(q_0^1, \varepsilon)}) = L(\mathcal{A}_2^{(q_0^2, \varepsilon)})$ where (q_0^1, ε) and (q_0^2, ε) are the initial configurations of $\mathcal{A}_1$ and $\mathcal{A}_2$ respectively.

Theorem 18. *Both the configuration equivalence problem and the language equivalence problem are in PSPACE for DRAs over dense domain $(\Sigma, <)$.*

The configuration equivalence problem is referred to as the residual equivalence problem in [6], and is shown to be in PTIME for well-typed DRAs over unordered (identity) data domains $(\Sigma, =)$ [6, Theorem 8].

Given a k -DRA $\mathcal{A}$ over $(\Sigma, <)$, a word $u \in \Sigma^*$ and $a \in u$, the *memorability problem* asks whether a is L -memorable in u , that is, whether $a \in \text{mem}_L(u)$. We show this problem is in PSPACE, which improves upon the NEXPTIME upper-bound for the memorability problem established in [7, Proposition 3].

Theorem 19. *The memorability problem is in PSPACE for DRAs over dense domain $(\Sigma, <)$.*

Over a dense domain, memorability reduces to configuration equivalence: replacing a by a fresh value while preserving the order type yields two configurations that are inequivalent exactly when a is memorable.

Experiments. We implemented the proposed active learning algorithm in Python⁶ and evaluated it on the running examples in this paper as well as randomly generated DRAs. The results confirm that the numbers of membership, equivalence, and memorability queries grow polynomially with the size of the target automaton, in line with the theoretical bounds, with membership queries dominating in practice. We also compared our approach with RaLib [12]; RaLib uses tree-based queries with black-box approximate equivalence checking, whereas our method uses word-based queries with white-box exact equivalence checking. The experiments show that both approaches are effective in their respective settings. See Section E and Section F for more details.

5 Conclusion and Discussion

In this paper, we studied DRAs over ordered domains. We showed that, with respect to minimization, DRAs over dense and non-dense domains can be treated

⁶ [liyong31/RALearning.git](https://github.com/liyong31/RALearning.git)

within a unified framework. This yields decidability of the minimization problem for DRAs over non-dense domains, resolving a previously open question [7].

Building on this unified framework, we developed and implemented an active learning procedure for DRAs that makes use of membership, equivalence, and memorability queries. Our experimental results demonstrate that the learning framework presented in this paper complements existing approaches for DRAs over ordered domains.

A natural question arises: can memorability queries be removed to obtain an algorithm similar to the classic L^* ? We believe this is possible by using the notion of *abstract suffixes* introduced in [23]. Instead of treating the suffix v returned by Algorithm 1 as a concrete word, we treat it as an *abstract* suffix representing all possible suffixes z for a prefix u over a set $D_{u,v}$ with $z \sim_R v$. Let v contain ℓ distinct letters. For each prefix $u = b_1 \dots b_m \in U$, let $b'_1 < \dots < b'_m$ be the sorted values in u , $b'_0 = b'_1 - 1$, and $b'_{m+1} = b'_m + 1$. Then, $D_{u,v} = \bigcup_{i=0}^m \bigcup_{j=1}^{\ell+1} \{b'_i + \frac{j}{\ell+1} \times (b'_{i+1} - b'_i)\}$. The suffixes z cover all possible extensions of u that have the same word type as v , analogous to our previous computation for the one-letter extensions of the memorable word $M(u)$. Since $M(u)$ is unknown, we have to consider all possible suffixes of the same word type to infer $M(u)$, instead of a single suffix v from a counterexample.

Checking S -consistency and the corresponding counterexample analysis procedures also become more involved. We leave a full learning algorithm using only membership and equivalence queries as future work. Unsurprisingly, this approach requires an exponential number of membership queries, as also in [23], to fill the observation table since all concrete suffixes z over $D_{u,v}$ must be enumerated and added to the suffix set for u .

We also note that the register automata underlying RaLib operate on data words over $A \times D$, where A is a finite action alphabet and D is an infinite data domain, whereas our model considers words directly over D . Hence, the two models are not directly equivalent, since the action-labelled model can distinguish inputs carrying the same data value but different actions. A natural extension of the model considered here is to $A \times D$, with word types preserving the finite action sequence while comparisons and renamings apply only to the data components. We conjecture that this action-extended variant has the same expressive power as the corresponding action-labelled register automata. Establishing a formal correspondence between the two models, and determining whether our learning and complexity bounds carry over to this extension, are interesting directions for future work.

Acknowledgements. We would like to thank the anonymous reviewers for their suggestions that helped improve the paper. This work was supported in part by the National Key R&D Program of China (Grant No. 2025YFE0220300), the Beijing Natural Science Foundation Project No. IS26039, CAS Project for Young Scientists in Basic Research (Grant No. YSBR-040), and the Engineering and Physical Sciences Research Council (EPSRC) through grant EP/X042596/1.

 This project is part of the European Union's Horizon Europe programme under the Marie Skłodowska-Curie grant agreement No. 101208673.

References

1. Alur, R., D’Antoni, L., Deshmukh, J., Raghothaman, M., Yuan, Y.: Regular functions and cost register automata. In: LICS. pp. 13–22 (2013)
2. Angluin, D.: Learning regular sets from queries and counterexamples. *Inf. Comput.* **75**(2), 87–106 (1987)
3. Argyros, G., D’Antoni, L.: The learnability of symbolic automata. In: Chockler, H., Weissenbacher, G. (eds.) *Computer Aided Verification*. pp. 427–445. Springer International Publishing, Cham (2018)
4. Argyros, G., Stais, I., Kiayias, A., Keromytis, A.D.: Back in black: Towards formal, black box analysis of sanitizers and filters. In: S&P. pp. 91–109 (2016)
5. Balachander, M., Filiot, E., Gentilini, R.: Passive learning of regular data languages in polynomial time and data. In: Majumdar, R., Silva, A. (eds.) *CONCUR. LIPIcs*, vol. 311, pp. 10:1–10:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2024)
6. Balachander, M., Filiot, E., Gentilini, R., Tzevelekos, N.: Register automata with permutations. In: Gawrychowski, P., Mazowiecki, F., Skrzypczak, M. (eds.) *MFCFS. LIPIcs*, vol. 345, pp. 14:1–14:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2025)
7. Benedikt, M., Ley, C., Puppis, G.: Minimal memory automata. Tech. rep. (2010), long version of ‘What You Must Remember When Processing Data Words’.
8. Bojańczyk, M., Klin, B., Lasota, S.: Automata theory in nominal sets. *Logical Methods in Computer Science* **Volume 10, Issue 3**, 4 (2014)
9. Bollig, B., Habermehl, P., Leucker, M., Monmege, B.: A fresh approach to learning register automata. In: Béal, M.P., Carton, O. (eds.) *DLT*. pp. 118–130. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
10. Cassel, S., Falk, H., Jonsson, B.: Ralib : A learnlib extension for inferring efsms. In: *DIFTS* (2015)
11. Cassel, S., Howar, F., Jonsson, B., Merten, M., Steffen, B.: A succinct canonical register automaton model. *Journal of Logical and Algebraic Methods in Programming* **84**(1), 54–66 (2015)
12. Cassel, S., Howar, F., Jonsson, B., Steffen, B.: Active learning for extended finite state machines. *Formal Aspects Comput.* **28**(2), 233–263 (2016)
13. Chen, Y., Hong, C., Lin, A.W., Rümmer, P.: Learning to prove safety over parameterised concurrent systems. In: Stewart, D., Weissenbacher, G. (eds.) *FMCAD*. pp. 76–83. IEEE (2017)
14. Chen, Y.F., Lengal, O., Tan, T., Wu, Z.: Register automata with linear arithmetic . In: LICS. pp. 1–12. IEEE Computer Society, Los Alamitos, CA, USA (Jun 2017)
15. Cobleigh, J.M., Giannakopoulou, D., Pasareanu, C.S.: Learning assumptions for compositional verification. In: Garavel, H., Hatcliff, J. (eds.) *TACAS. LNCS*, vol. 2619, pp. 331–346. Springer (2003)
16. Czerny, M.X.: Learning-based software testing: Evaluation of Angluin’s L^* algorithm and adaptations in practice (2014)
17. D’Antoni, L., Veanes, M.: The power of symbolic automata and transducers. In: Majumdar, R., Kunčák, V. (eds.) *Computer Aided Verification*. pp. 47–67. Springer International Publishing, Cham (2017)
18. Dierl, S., Fiterau-Brostean, P., Howar, F., Jonsson, B., Sagonas, K., Tåquist, F.: Scalable tree-based register automata learning. In: Finkbeiner, B., Kovács, L. (eds.) *TACAS. LNCS*, vol. 14571, pp. 87–108. Springer (2024)
19. Drews, S., D’Antoni, L.: Learning symbolic automata. In: Legay, A., Margaria, T. (eds.) *TACAS*. pp. 173–189. Springer, Berlin, Heidelberg (2017)

20. Fisman, D., Frenkel, H., Zilles, S.: Inferring symbolic automata. *Logical Methods in Computer Science* **Volume 19, Issue 2**, 5 (2023)
21. Goles, E., Montealegre, P., Salo, V., Törmä, I.: Pspace-completeness of majority automata networks. *Theor. Comput. Sci.* **609**, 118–128 (2016)
22. Grinchtein, O., Jonsson, B., Leucker, M.: Learning of event-recording automata. *Theoretical Computer Science* **411**(47), 4029–4054 (2010)
23. Howar, F., Steffen, B., Jonsson, B., Cassel, S.: Inferring canonical register automata. In: Kuncak, V., Rybalchenko, A. (eds.) *VMCAI*. LNCS, vol. 7148, pp. 251–266. Springer (2012)
24. Kaminski, M., Francez, N.: Finite-memory automata. *Theoretical Computer Science* **134**(2), 329–363 (1994)
25. Ley, C.: Forward looking logics and automata (2011)
26. Maler, O., Mens, I.E.: A Generic Algorithm for Learning Symbolic Automata from Membership Queries, pp. 146–169. Springer International Publishing (2017)
27. Mens, I.E., Maler, O.: Learning regular languages over large ordered alphabets. *Logical Methods in Computer Science* **Volume 11, Issue 3**, 13 (2015)
28. Moerman, J., Sammartino, M., Silva, A., Klin, B., Szynwelski, M.: Learning nominal automata. In: Castagna, G., Gordon, A.D. (eds.) *POPL*. pp. 613–625. ACM (2017)
29. Murawski, A.S., Ramsay, S.J., Tzevelekos, N.: Polynomial-time equivalence testing for deterministic fresh-register automata. In: Potapov, I., Spirakis, P.G., Worrell, J. (eds.) *MFCS. LIPIcs*, vol. 117, pp. 72:1–72:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2018)
30. Myhill, J.: Finite automata and the representation of events. In: Technical Report WADD TR-57-624. p. 112–137 (1957)
31. Neider, D., Smetters, R., Vaandrager, F., Kuppens, H.: Benchmarks for Automata Learning and Conformance Testing, pp. 390–416. Springer International Publishing, Cham (2019)
32. Nerode, A.: Linear automaton transformations. In: American Mathematical Society. p. 541–544 (1958)
33. Neven, F., Schwentick, T., Vianu, V.: Finite state machines for strings over infinite alphabets. *ACM Trans. Comput. Logic* **5**(3), 403–435 (2004)
34. Ozaki, A.: Actively learning from machine learning models with queries and counterexamples (extended abstract). In: Endriss, U., Melo, F.S., Bach, K., Diz, A.J.B., Alonso-Moral, J.M., Barro, S., Heintz, F. (eds.) *ECAI. Frontiers in Artificial Intelligence and Applications*, vol. 392, pp. 25–26. IOS Press (2024)
35. Royden, H., Fitzpatrick, P.: *Real Analysis*. Prentice Hall (2010)
36. de Ruiter, J., Poll, E.: Protocol state fuzzing of TLS implementations. In: Jung, J., Holz, T. (eds.) *USENIX*. pp. 193–206. USENIX Association (2015)
37. Shih, A., Darwiche, A., Choi, A.: Verifying binarized neural networks by Angluin-style learning. In: Janota, M., Lynce, I. (eds.) *SAT*. LNCS, vol. 11628, pp. 354–370. Springer (2019)
38. Vaandrager, F.W., Garhewal, B., Rot, J., Wißmann, T.: A new approach for active automata learning based on apartness. In: Fisman, D., Rosu, G. (eds.) *TACAS*. LNCS, vol. 13243, pp. 223–243. Springer (2022)
39. Venhoek, D., Moerman, J., Rot, J.: Fast computations on ordered nominal sets. *Theor. Comput. Sci.* **935**, 82–104 (2022)
40. Weiss, G., Goldberg, Y., Yahav, E.: Extracting automata from recurrent neural networks using queries and counterexamples. In: Dy, J.G., Krause, A. (eds.) *ICML*. PMLR, vol. 80, pp. 5244–5253. PMLR (2018)

A Missing Proofs of Section 3

A.1 Proof of Theorem 7

Theorem 7. *The configuration reachability problem for DRAs over $(\mathbb{Z}, <)$ is decidable.*

Proof. Let $\mathcal{A} = (Q, q_0, F, \Delta)$ be a well-typed DRA over $(\mathbb{Z}, <)$ with κ registers, (p, v) be a configuration and q a state, and let $\min(v)$ and $\max(v)$ denote the smallest and largest integers in v , respectively. Suppose $\pi = (p, v) \xrightarrow{u} (q, w)$ for some words u, w , where the step-by-step transitions of π are:

$$(p_0, v_0) \xrightarrow{a_1} (p_1, v_1) \xrightarrow{a_2} \dots \xrightarrow{a_n} (p_n, v_n).$$

The statement of the theorem holds if the following claims are true:

(C1) We can obtain a run π_{squeeze} from π , where

$$\pi_{\text{squeeze}} = (p_0, x_0) \xrightarrow{b_1} (p_1, x_1) \xrightarrow{b_2} \dots \xrightarrow{b_n} (p_n, x_n)$$

for some $x_0, \dots, x_n$ and $b_1, \dots, b_n$. Moreover, the range of the values occurring in $x_0, \dots, x_n$ is bounded between $\min(v) - 3\kappa \cdot n$ and $\max(v) + 3\kappa \cdot n$.

(C2) If $n > (\sum_{i=0}^{\kappa} (\max(v) - \min(v) + 2)^i) \cdot |Q| \equiv \text{len}(\mathcal{A}, v)$, then there exists a run π_{short} from (p, v) to (q, w') for some w' with $|\pi_{\text{short}}| < |\pi|$.

With (C1) and (C2), to verify whether there exists a run from (p, v) to (q, w) for some w , it suffices to examine runs of length at most $\text{len}(\mathcal{A}, v)$ and over configurations in $Q \times X$, where X consists of words of length between 0 and κ and over symbols in $\{\min(v) - \text{len}(\mathcal{A}, v), \dots, \max(v) + \text{len}(\mathcal{A}, v)\} \subset \mathbb{Z}$. Since there are only finitely many such runs, the verification process is decidable.

To derive (C1) and (C2), we show that one can effectively obtain an order-preserving mapping σ such that $\sigma(v) = v$ and

$$\pi' = (p, \sigma(v)) \xrightarrow{\sigma(u)} (q, \sigma(w))$$

is a valid run satisfying the following two properties: (P1) If a value between $\min(v) - 3\kappa \cdot n$ and $\max(v) + 3\kappa \cdot n$ is missing in u , then $\max(\sigma(u)) - \min(\sigma(u)) < \max(u) - \min(u)$. (P2) If $n > \text{len}(\mathcal{A}, v)$, then a subrun of π' can be removed while the remaining part is still a valid run, a property analogous to the pumping lemma for DFAs.

Consider (C1). Suppose there exist three values a , a' , and a'' , satisfying $\max(v) < a < a + 1 = a' < a' + 1 = a'' \leq \max(v) + 3\kappa \cdot n$, such that none of a , a' , or a'' appear in any v_i for $i \geq 1$. Let σ_a^{shift} be the order-preserving partial mapping that maps each value b to itself if $b < a$, and to $b - d$ if $b > a''$, where d is the difference between a'' and the minimum value greater than a'' among all components in $v_1, \dots, v_n$ and $a_1, \dots, a_n$. To ensure that σ_a^{shift} is well-defined, we also require $\sigma_a^{\text{shift}}(b) = a'$ for $b \in \{a, a', a''\}$. Then $\sigma_a^{\text{shift}}(v) = v$, and

$$(p_0, \sigma_a^{\text{shift}}(v_0)) \xrightarrow{\sigma_a^{\text{shift}}(a_1)} \dots \xrightarrow{\sigma_a^{\text{shift}}(a_n)} (p_n, \sigma_a^{\text{shift}}(v_n))$$

is still a valid run from (p, v) to (q, w') for some w' . Moreover, the range of the values occurring in $\sigma_a^{shift}(v_1), \dots, \sigma_a^{shift}(v_n)$ is strictly smaller than the range of $v_1, \dots, v_n$. This technique applies symmetrically when $\min(v) - 3\kappa \cdot n \leq a < \min(v)$. By repeatedly applying this process at most $\kappa \cdot n$ times, we obtain the run π_{squeeze} satisfying (C1).

Next, consider (C2). By the pigeonhole principle, if $|\pi| > |Q|$, then along the run there must be a repeated state. Hence, there exists a decomposition $u = u_1 u_2 u_3$ and a state r such that

$$\pi = (p, v) \xrightarrow{u_1} (r, x) \xrightarrow{u_2} (r, y) \xrightarrow{u_3} (q, w) \quad (1)$$

for some words x, y . We could immediately delete the subrun $(r, x) \xrightarrow{u_2} (r, y)$ if $x = y$, and the remaining run $(p, v) \xrightarrow{u_1} (r, x) \xrightarrow{u_3} (q, w)$ would still be valid. However, in general we do not have $x = y$.

More generally, for each $\ell \in \mathbb{N}$, if $|\pi| > \ell \cdot |Q|$, we obtain a decomposition

$$\pi = (p, v) \xrightarrow{u_1} (r, x_1) \xrightarrow{u_2} (r, x_2) \dots \xrightarrow{u_\ell} (r, x_\ell) \xrightarrow{u_{\ell+1}} (q, w)$$

for some $x_1, \dots, x_\ell$ and $u_1, \dots, u_{\ell+1}$. For each i , let y_i be the sorted version of x_i . Then $y_1, \dots, y_\ell$ are strictly increasing sequences of the same length θ for some θ . Let $y_i = a_1^{[i]} \dots a_\theta^{[i]}$ for each $i = 1, \dots, \ell$. If there exist two distinct indices $i < j$ such that, for every $s = 1, \dots, \theta - 1$,

$$a_{s+1}^{[i]} - a_s^{[i]} \geq a_{s+1}^{[j]} - a_s^{[j]}, \quad (2)$$

then by letting σ be the mapping $\sigma(a) =$

- $a + (a_1^{[i]} - a_1^{[j]})$ if $a < a_1^{[j]}$.
- $a + (a_s^{[i]} - a_s^{[j]})$ if $a_s^{[j]} \leq a \leq a_{s+1}^{[j]}$ for $s = 1, \dots, \theta - 1$.
- $a + (a_\theta^{[i]} - a_\theta^{[j]})$ if $a \geq a_\theta^{[j]}$.

Hence, we have that σ is order-preserving, and

$$(r, \sigma(x_j)) \xrightarrow{\sigma(u_j)} (r, \sigma(x_{j+1})) \xrightarrow{\sigma(u_{j+1})} \dots (r, \sigma(x_\ell)) \xrightarrow{\sigma(u_{\ell+1})} (q, \sigma(w))$$

is a valid run from $(r, \sigma(x_j))$ to $(q, \sigma(w))$ on $\sigma(u_j \dots u_{\ell+1})$, where $\sigma(x_j) = x_i$. Accordingly,

$$(p, v) \xrightarrow{u_1} \dots (r, x_i) \xrightarrow{\sigma(u_j)} (r, \sigma(x_{j+1})) \xrightarrow{\sigma(u_{j+1})} \dots (r, \sigma(x_\ell)) \xrightarrow{\sigma(u_{\ell+1})} (q, \sigma(w))$$

is a shorter valid run from (p, v) to $(q, \sigma(w))$.

The following property of finite sequences will be used later:

Lemma 20. *Let $\delta_1, \dots, \delta_\ell$ be ℓ pairwise distinct strictly increasing sequences over $\mathbb{N}$, where each $\delta_i = d_1^{[i]} \dots d_\theta^{[i]}$ for some fixed θ . Given a constant $d \in \mathbb{N}$, if $\ell > (d + 2)^\theta$, then there exist two distinct indices i, j such that for every s , both of values $d_s^{[i]}$ and $d_s^{[j]}$ are greater than d whenever they are not equal.*

Proof. Consider the sequences over $\{0, 1, \dots, d, d+1\}$ of length θ . There are exactly $(d+2)^\theta$ such sequences. Among those sequences, there is at least one over exactly $\{0, d+1\}$. Given two distinct δ_i and δ_j , we let $e_{i,j} = f_1 \dots f_\theta$ a sequence over $\{0, 1, \dots, d, d+1\}$, where $f_s = 0$ if $d_s^{[i]} = \delta_s^{[j]}$; $= t$ if $d_s^{[i]} \neq \delta_s^{[j]}$ and $\min(d_s^{[i]}, \delta_s^{[j]}) = t$; $= d+1$, otherwise. If $\ell > (d+2)^\theta$, there exists $e_{i,j}$ over exactly $\{0, d+1\}$. Accordingly, if $\ell > (d+2)^\theta$ there are at least two δ_i and δ_j such that $\delta_i \neq \delta_j$ and for each s , both of values $d_s^{[i]}$ and $d_s^{[j]}$ are greater than d whenever they are not equal. $\square$

Now assume that for every distinct pair of i and j , there exists s such that

$$a_{s+1}^{[i]} - a_s^{[i]} \neq a_{s+1}^{[j]} - a_s^{[j]}. \quad (3)$$

By Lemma 20, if $\ell > (\max(v) - \min(v) + 2)^\theta$, then there exist indices i, j such that both differences $a_{s+1}^{[i]} - a_s^{[i]}$ and $a_{s+1}^{[j]} - a_s^{[j]}$ exceed $\max(v) - \min(v)$ whenever, for every distinct pair i, j , there exists an index s satisfying Eq. 3.

We define ι as an index between 1 and θ , where:

- $\iota = 1$ if $a_1^{[i]} > \max(v)$,
- $\iota = \theta$ if $a_\theta^{[i]} < \min(v)$,
- otherwise, ι is the smallest index satisfying $\min(v) \leq a_\iota^{[i]} \leq \max(v)$.

Let $d_1 \dots d_{\theta-1}$ be the sequence defined by

$$d_s = \max\{a_{s+1}^{[i]} - a_s^{[i]}, a_{s+1}^{[j]} - a_s^{[j]}\}$$

for all $s = 1, \dots, \theta - 1$. We define σ as follows:

- $\sigma(a) = a + a_\iota^{[i]} + (d_\iota + \dots + d_{s-1})$ if $a_s^{[i]} \leq a \leq a_{s+1}^{[i]}$ and $s \geq \iota$,
- $\sigma(a) = a + a_\iota^{[i]} - (d_{\iota-1} + \dots + d_{s+1})$ if $a_s^{[i]} \leq a \leq a_{s+1}^{[i]}$ and $s < \iota$.

Then the following properties hold:

- σ is order-preserving,
- $\sigma(a_{s+1}^{[i]}) - \sigma(a_s^{[i]}) = d_s$ for all s ,
- $\sigma(v) = v$, since there is at most one index s satisfying Eq. 3 and for that index we have $\min(v) \leq a_s^{[i]} \leq \max(v)$.

Accordingly,

$$\pi_{\text{pre}} = (p, \sigma(v)) \xrightarrow{\sigma(u_1)} \dots \xrightarrow{\sigma(u_i)} (r, \sigma(x_i))$$

is a valid run from (p, v) to $(r, \sigma(x_i))$ on $\sigma(u_1) \dots \sigma(u_i)$. Hence, π_{pre} together with the run $(r, x_j) \xrightarrow{u_{j+1} \dots u_{\ell+1}} (q, w)$ satisfies Eq. 2. $\square$

Algorithm 3: Minimization algorithm for DRAs

```

input: A DRA  $\mathcal{A} = (Q, q_0, F, \Delta)$  recognizing  $L$  over  $(\Sigma, R)$ 
1 for  $q \in Q$  do
2   | Obtain a word  $w_q$  such that  $(q_0, \epsilon) \xrightarrow{w_q}_{\mathcal{A}} (q, w)$  for some  $w$ ;
3 end
4  $flag \leftarrow \top$ ;
5 while  $flag$  do
6   |  $flag \leftarrow \perp$ ;
7   for  $p \in Q$  do
8     | for  $q \in Q \setminus \{p\}$  do
9       | if  $w_p \cong_L w_q$  then
10        | | Merge  $p$  and  $q$ .;
11        | |  $flag \leftarrow \top$ ;
12        | end
13      | end
14    | end
15 end
16 for  $q \in Q$  do
17   | Modify transitions starting from  $q$ .;
18 end
19 return  $\mathcal{A}$ ;

```

A.2 Minimization Algorithm for DRAs

By Theorem 6, the minimization problem for DRAs is decidable, provided that the canonical automaton of every given DRA can be effectively constructed. For DFAs, the canonical automaton can be obtained by merging states that correspond to the same residual language. The same idea applies to DRAs, as guaranteed by the following lemma:

Lemma 21 ([7]). *Let $\mathcal{A}$ be a DRA recognizing L . For any two words u, v , if $(q_0, \epsilon) \xrightarrow{u} (q, x)$ and $(q_0, \epsilon) \xrightarrow{v} (q, y)$ for some configurations (q, x) and (q, y) , then $u \cong_L v$.*

The pseudocode for DRA minimization is given in Algorithm 3. In the algorithm, we first compute a representative word w_q for each state q . Then, for any pair of states p, q , we merge them whenever $w_p \cong_L w_q$. After this step, the automaton is already state-minimal. However, it may still fail to be data-minimal. Therefore, in a final step, we use $mem_L(w_q)$ for each state q to adjust transitions outgoing from q so as to achieve data-minimality.

The decidability of the following three problems ensures that Algorithm 3 is an effective minimization procedure:

- **Representative problem:** Given a state q , compute a representative word w_q such that $(q_0, \epsilon) \xrightarrow{w_q}_{\mathcal{A}} (q, w)$ for some word w , whenever such a representative exists.

- **Memorability problem:** Given a word u , compute $\text{mem}_L(u)$.
- **Word-equivalence problem:** Given two words u, v , verify whether $u \cong_L v$.

Clearly, the representative problem is decidable if the configuration reachability problem is decidable and a witness word w_q can be effectively extracted.

As for DFAs, the equivalence problem of two DRAs reduces to the reachability problem for their product automaton.

Definition 22. *Given two DRA $\mathcal{A}_1$ and $\mathcal{A}_2$, the product automaton $\mathcal{A}_1 \times \mathcal{A}_2$ is a tuple $(Q^\times, q_0^\times, F^\times, \Delta^\times)$ where:*

- $Q^\times = Q^1 \times Q^2$ is a set of states.
- $(q_0^1, q_0^2) \in Q^\times$ is the initial state.
- $F^\times = F^1 \times F^2$ is the set of final states.
- $\Delta^\times$ is a finite set of transitions of the form $((p_1, p_2), (\tau_1, \tau_2), (E_1, E_2), (q_1, q_2))$, where $(p_i, \tau_i, E_i, q_i) \in \Delta^i$ for $i \in \{1, 2\}$.

This product register automaton differs from the RAs in Definition 1: it is not well-typed, since its registers may contain duplicate values. However, it can be translated into a DRA satisfying Definition 1 by introducing additional states that simulate the register configuration so as to prevent duplicate values. Thus, the equivalence problem for DRAs from given configurations is decidable whenever the configuration reachability problem is decidable.

Given a word u , if $(q_0, \epsilon) \xrightarrow{u}_{\mathcal{A}} (q, w)$, then $\text{mem}_L(u)$ is a subsequence of w , obtained by deleting all non-memorable symbols. If every extension run $(q, w) \xrightarrow{s} (r, x)$ from (q, w) satisfies $r \notin F$, then $\text{mem}_L(u) = \epsilon$. Otherwise, suppose $(q, w) \xrightarrow{s} (r, x)$ for some $r \in F$, and let $w' = \sigma(w)$ for some order preserving mapping σ which maps a to a different value b . If the DRA started from (q, w) is not equivalent to that started from (q, w') , then a is memorable; otherwise, it is not. When Σ is dense, such a w' always exists. This need not hold for non-dense alphabets. For instance, if $w = 2 \cdot 3 \cdot 4$ over $\mathbb{Z}$ and $a = 3$, then there is no $b \neq 3$ such that σ preserves the ordering relations in w . In contrast, if $w = 1 \cdot 3 \cdot 5$ and $a = 3$, we may choose $b = 4$.

Recall that w arises from a run $(q_0, \epsilon) \xrightarrow{u} (q, w)$, and if $u \cong_L u'$ with $(q_0, \epsilon) \xrightarrow{u'} (q, w')$, then the i th symbol of w is memorable in u iff the i th symbol of w' is memorable in u' . Since L is a data language, for every $u \sim_R u'$, we have $u \cong_L u'$. For any sequence u , we can effectively compute a sequence u' such that $u' \sim_R u$ and any two distinct symbols in u' differ by more than 1. For example, if $u = 1 \cdot 2 \cdot 3$, we may take $u' = 1 \cdot 3 \cdot 5$. Hence the memorability problem reduces to the configuration equivalence problem, and thereby to the configuration reachability problem.

Suppose there exists an R -preserving mapping σ such that $\sigma(\text{mem}_L(u)) = \text{mem}_L(v)$ for given words u and v , and let $u' = \sigma(u)$. Then $u \cong_L v$ iff for all words s , $u' \cdot s \in L \iff v \cdot s \in L$. Let $(q_{u'}, w_{u'})$ and (q_v, w_v) be the configurations reached from (q_0, ϵ) upon reading u' and v , respectively. Therefore, $u \cong_L v$ iff the DRA $\mathcal{A}$ starting from $(q_{u'}, w_{u'})$ is equivalent to the one starting from (q_v, w_v) . Consequently, the word-equivalence problem, and thus the minimization

problem for DRAs, is decidable because the configuration reachability problem is decidable.

B More Examples on the Learning Algorithm

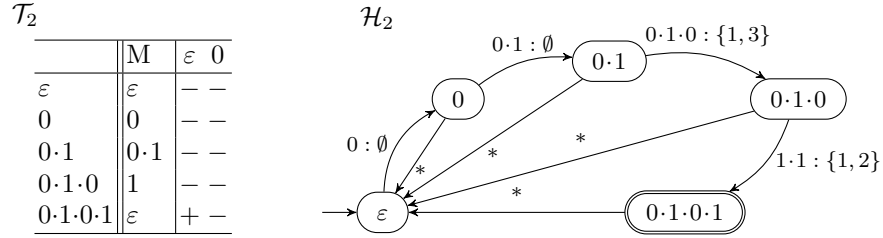


Fig. 4: Closed Table $\mathcal{T}_2$ and its derived DRA $\mathcal{H}_2$

Example 23. Let us reconsider the examples in Fig. 3. Assume that the teacher returns counterexample $w = 4 \cdot 5 \cdot 4 \cdot 5$ to the learner on equivalence query $\text{EQ}(\mathcal{H}_1)$. Clearly, $w \in L$ but $w \notin L(\mathcal{H}_1)$. Initially, we have $u = \varepsilon$, $r = \varepsilon$ and $mq = +$. In the first iteration, we decompose $w = a \cdot z$ with $a = 4$ and $z = 5 \cdot 4 \cdot 5$. Reading $a = 4$, the transition $(\varepsilon, \varepsilon) \xrightarrow{0:\emptyset} (0, 4)$ is taken when reading a , so $b = 0$. By applying σ_1 which maps 4 to 0, we obtain $\sigma_1(z) = 1 \cdot 0 \cdot 1$. Furthermore, σ_2 maps $M(u \cdot b) = 0$ to $M(u') = M(0) = 0$, hence σ is an identity map. Thus, $\sigma_2(\sigma_1(z)) = 1 \cdot 0 \cdot 1$ and $\text{MQ}(u' \cdot \sigma_2(\sigma_1(z))) = \text{MQ}(0 \cdot 1 \cdot 0 \cdot 1) = +$. We then set $u = 0$, $r = 0$ and $w = 1 \cdot 0 \cdot 1$ and go to next iteration. In the second iteration, we again write $w = a \cdot z$ with $a = 1$ and $z = 0 \cdot 1$. The transition $(0, 0) \xrightarrow{0 \cdot 1:\emptyset} (0 \cdot 1, 0 \cdot 1)$ is taken. Since $M(u) \cdot a = M(u) \cdot b = M(u')$, σ_1 and σ_2 are both identity mappings. Thus, $\sigma_2(\sigma_1(z)) = 0 \cdot 1$ and $\text{MQ}(u' \cdot \sigma_2(\sigma_1(0 \cdot 1))) = \text{MQ}(0 \cdot 1 \cdot 0 \cdot 1) = +$. We set $u = 0 \cdot 1$, $r = 0 \cdot 1$ and $w = 0 \cdot 1$ and go to the next iteration. In this last iteration, the same reasoning applies: $a = 0$ and $z = 1$ and the transition $(0 \cdot 1, 0 \cdot 1) \xrightarrow{0 \cdot 1 \cdot 0:\{1,3\}} (0 \cdot 1 \cdot 0, 1)$ is taken; so $b = 0$. Since $a = b$ and thus $M(u) \cdot a = M(u) \cdot b$, σ_1 is again the identity mapping. We have $M(u \cdot b) = M(0 \cdot 1 \cdot 0) = 1$ and $M(u') = M(0) = 0$, giving $v = \sigma_2(\sigma_1(z)) = 0$. Thus, $\text{MQ}(u' \cdot v) = \text{MQ}(0 \cdot 0) = -$. Therefore the returned word is $v = 0$ which will be added into S . After closing the updated table, we will obtain the table $\mathcal{T}_2$ and derive a new conjecture DRA $\mathcal{H}_2$, as shown in Fig. 4. After refining $\mathcal{H}_2$ with counterexample $1 \cdot 0 \cdot 1 \cdot 2 \cdot 1 \cdot 2$, we will get the final table $\mathcal{T}$ and construct from $\mathcal{T}$ the correct conjecture DRA $\mathcal{A}$ in Fig. 2. $\square$

C More Details on Learning DRAs over Non-dense Domains

We mentioned in the main content that one can learn DRAs over $(\mathbb{Z}, <)$ with our DRA learner for $(\mathbb{Q}, <)$. Here we provide more details on that.

Assume that the teacher answers only membership and memorability queries on words over $\mathbb{Z}$, and equivalence query over a DRA over $\mathbb{Z}$. Let $L_{\mathbb{Z}}$ be our target language over $\mathbb{Z}$ and $L_{\mathbb{Q}}$ be its corresponding language over $\mathbb{Q}$. We can just add an interface between our DRA learner of dense domains and the teacher for non-dense domains.

- For asking a membership query for a word u over $\mathbb{Q}$, we can use a bijective order-preserving mapping σ that maps a word over $\mathbb{Q}$ to a word over $\mathbb{Z}$. For instance, when $u = 1 \cdot 2 \cdot \frac{1}{2}$, we have $\sigma(u) = 2 \cdot 4 \cdot 1$. We then return the response on $\text{MQ}(\sigma(u))$. Since $u \sim_R \sigma(u)$, they should have the same membership in the target language $L_{\mathbb{Q}}$.
- For asking a memorability query for a word u over $\mathbb{Q}$, we also ask the memorability query $\text{Mem}(\sigma(u))$ instead and return $\sigma^{-1}(\text{Mem}(\sigma(u)))$. Since $u \sim_R \sigma(u)$ over the dense domains, we know that the memorable words for $\sigma(u)$ have the same positions as the memorable words for u .
- To resolve the equivalence query on the conjectured DRA $\mathcal{H}$ over $\mathbb{Q}$, we convert it to a DRA $\mathcal{H}'$ over $\mathbb{Q}$ for all word types of transitions using σ and ask equivalence query $\text{EQ}(\mathcal{H}')$. If the answer is “Yes”, we output the DRA $\mathcal{H}'$ over $\mathbb{Z}$, otherwise, just return the counterexample w to our DRA learner, since it is also a valid counterexample over $\mathbb{Q}$.

By Theorem 9, this learning algorithm will output a correct DRA of the target language $L_{\mathbb{Z}}$.

D Missing Proofs of Section 4.1

Lemma 11. *Let Σ be a dense set. For two words $u_1, u_2 \in \Sigma^*$ with $\text{mem}_L(u_1) \sim_R \text{mem}_L(u_2)$, $u_1 \not\equiv_L u_2$ if and only if there exist a bijective order-preserving mapping σ and a word w such that $\sigma(\text{mem}_L(u_1)) = \text{mem}_L(u_2)$ and $\sigma(u_1) \cdot w \in L \not\equiv u_2 \cdot w \in L$.*

Proof. We first prove the “only if” direction. Suppose that $u_1 \not\equiv_L u_2$. Since $\text{mem}_L(u_1) \sim_R \text{mem}_L(u_2)$, by Definition 5, there exist words x, y such that $\text{mem}_L(u_1) \cdot x \sim_R \text{mem}_L(u_2) \cdot y$, but exactly one of the words $u_1 \cdot x$ and $u_2 \cdot y$ is in L . Because Σ is dense, the relation $\text{mem}_L(u_1) \cdot x \sim_R \text{mem}_L(u_2) \cdot y$ implies the existence of an order-preserving mapping σ such that $\sigma(\text{mem}_L(u_1) \cdot x) = \text{mem}_L(u_2) \cdot y$. By letting $w = \sigma(x) = y$, we obtain that either $\sigma(u_1) \cdot w \in L$ or $u_2 \cdot w \in L$.

Next, we prove the “if” direction. Suppose there exist a word w and an order-preserving mapping σ such that $\sigma(\text{mem}_L(u_1)) = \text{mem}_L(u_2)$ and $\sigma(u_1) \cdot w \in L \not\equiv u_2 \cdot w \in L$. Let $x = \sigma^{-1}(w)$ and $y = w$. Since σ is order-preserving and Σ

is dense, we have $\text{mem}_L(u_1) \cdot x \sim_R \text{mem}_L(u_2) \cdot y$. Moreover, exactly one of the words $u_1 \cdot x$ and $u_2 \cdot y$ is in L . Hence, $u_1 \not\equiv_L u_2$. $\square$

To order to prove Lemma 12, it suffices to prove the following Lemma 24, since Lemma 12 is a direct consequence of Lemma 24.

Lemma 24. *Let L be a data language over (Σ, R) where Σ is dense. For a word $u \in \Sigma^*$ and two bijective order-preserving mappings σ, σ' with $\sigma(\text{mem}_L(u)) = \sigma'(\text{mem}_L(u))$, it holds that $\sigma(u) \cdot w \in L \Leftrightarrow \sigma'(u) \cdot w \in L$ for all $w \in \Sigma^*$.*

Proof. Since σ and σ' are bijective, their inverses σ^{-1} and σ'^{-1} exist and are also order-preserving. Let u be a word. By the assumption $\sigma(\text{mem}_L(u)) = \sigma'(\text{mem}_L(u))$, it follows that for every word $w \in \Sigma^*$, $\text{mem}_L(u) \cdot \sigma^{-1}(w) \sim_R \sigma(\text{mem}_L(u) \cdot \sigma^{-1}(w)) = \sigma(\text{mem}_L(u)) \cdot w = \sigma'(\text{mem}_L(u)) \cdot w \sim_R \sigma'^{-1}(\sigma'(\text{mem}_L(u)) \cdot w) = \text{mem}_L(u) \cdot \sigma'^{-1}(w)$ holds. That is, $\text{mem}_L(u) \cdot \sigma^{-1}(w) \sim_R \text{mem}_L(u) \cdot \sigma'^{-1}(w)$ holds for every $w \in \Sigma^*$. By definition of $\cong_L$, since $\text{mem}_L(u) \cdot \sigma^{-1}(w) \sim_R \text{mem}_L(u) \cdot \sigma'^{-1}(w)$, we have $u \cdot \sigma^{-1}(w) \in L \Leftrightarrow u \cdot \sigma'^{-1}(w) \in L$ for all $w \in \Sigma^*$ because $u \cong_L u$ holds. It then follows that, $\sigma(u) \cdot w \in L \Leftrightarrow \sigma'(u) \cdot w \in L$ for all $w \in \Sigma^*$ since σ and σ' are order-preserving mappings. $\square$

Lemma 12. *Let Σ be a dense set, and let u_1, u_2 be distinct words. If σ is a bijective order-preserving mapping with $\sigma(\text{mem}_L(u_1)) = \text{mem}_L(u_2)$, then $u_1 \not\equiv_L u_2$ iff there exists $w \in \Sigma^*$ such that $\sigma(u_1) \cdot w \in L \not\equiv u_2 \cdot w \in L$.*

Proof. It is a direct consequence of Lemma 24. $\square$

The following lemma, which captures the key properties of S -consistency, follows directly from its definition.

Lemma 25. *Let $S \subseteq \Sigma^*$ be a set of words. We have:*

1. *if u is not S -consistent with v , then $u \not\equiv_L v$;*
2. *if u is not S -consistent with v , then u is not S -consistent with v whenever $S \subseteq S'$.*

We present examples below to show that S -consistency is *not* an equivalence relation: it is neither symmetric nor transitive.

Example 26. Consider the language L_4 where $n = 4$ for the language in Example 2, that is, $L_4 = \{w \in \mathbb{Q} \mid |w| = 4 \text{ and } w \text{ is strictly increasing or decreasing}\}$.

	M	ε	$1 \cdot 2$
ε	ε	—	—
0	0	—	—
$0 \cdot 1$	1	—	—
$0 \cdot 1 \cdot 2$	2	—	—

During the learning process, the above observation table may arise. The prefixes of the first three rows belong to U , whereas the last row, $0 \cdot 1 \cdot 2$, belongs to X , the only one-letter extension in X shown in this table. We denote by u_i the prefix corresponding to the i th row. This table witnesses *non-symmetry*: u_2 is S -consistent with u_3 , but u_3 is not S -consistent with u_2 . In addition, the one-letter extension $0 \cdot 1 \cdot 2$ (in X) is S -consistent with both u_2 and u_3 , witnessing *non-transitivity* of S -consistency.

In order to prove Lemma 15, we first prove the following Lemma.

Lemma 27. *Let $w = a_1 \cdots a_d$ and $\pi = (u_0, r_0) \xrightarrow{a_1} \cdots \xrightarrow{a_d} (u_d, r_d)$ be the run of $\mathcal{H}$ on w . Let $u'_0, \dots, u'_d$ be the sequence of states visited in Algorithm 1 if we remove the **if** statement (i.e., we do not use the early return). It then holds that $u'_i = u_i$ for all $0 \leq i \leq d$.*

Proof. Let $r'_0, \dots, r'_d$ and $a'_1 z_0, \dots, a'_d z_{d-1}$ be the corresponding register values and the remaining suffixes visited in Algorithm 1 where $z_d = \varepsilon$ and $a'_1 z_0 = w$. Let $r'_0 b_1, \dots, r'_d b_d$ be the selected word type visited in Algorithm 1. We first prove the following invariants in the whole loop by induction:

- (i) $r'_i \cdot a'_{i+1} \sim_R M(u'_i) \cdot b_{i+1}$ for all $0 \leq i < d$.
- (ii) $r'_i a'_{i+1} z_i \sim_R r'_i b_{i+1} \sigma_1(z_i)$ for all $0 \leq i < d$.

For the base case, obviously, when $i = 0$, we have $u'_0 = r'_0$ and $z_0 = w[2 \cdots]$. It follows that $M(u'_0) = \varepsilon$ and thus $r'_0 \cdot a'_1 \sim_R M(u'_0) \cdot b_1$ holds directly. Moreover, σ_1 is in fact an order-preserving mapping from $a'_1 = a_1$ to b_1 . Hence, $\sigma_1(a'_1 z_0) = b_1 \cdot \sigma_1(z_0) \sim_R a'_1 z_0$.

For the induction step, assume that it holds for all $i \leq \ell$. According to Algorithm 1, $r'_{\ell+1} = M(u'_{\ell+1})$ and $z_{\ell+1} = \sigma_2(\sigma_1(z_\ell))$. Since when reading $a'_{\ell+2}$ at configuration $(u'_{\ell+1}, r'_{\ell+1} = M(u'_{\ell+1}))$, the transition with the word type $M(u'_{\ell+1}) \cdot b_{\ell+2}$ is selected, it indicates that $r'_{\ell+1} \cdot a'_{\ell+2} = M(u'_{\ell+1}) \cdot a'_{\ell+2} \sim_R M(u'_{\ell+1}) \cdot b_{\ell+2}$. Recall that current σ_1 is an order-preserving mapping from $r'_{\ell+1} \cdot a'_{\ell+2}$ to $r'_{\ell+1} \cdot b_{\ell+2}$. Therefore, it holds that $r'_{\ell+1} \cdot a'_{\ell+2} \cdot z_{\ell+1} \sim_R \sigma_1(r'_{\ell+1} \cdot a'_{\ell+2} \cdot z_{\ell+1}) = r'_{\ell+1} \cdot b_{\ell+2} \cdot \sigma_1(z_{\ell+1})$.

Since $r'_i a'_{i+1} z_i \sim_R r'_i b_{i+1} \sigma_1(z_i)$ for all $0 \leq i < d$, we have that $u'_i a'_{i+1} z_i \in L$ if and only if, $u'_i b_{i+1} \sigma_1(z_i) \in L$. The reason is the following. Assume that $\mathcal{A}$ is the minimal well-typed DRA of L defined by $\cong_L$. After reading u'_i , $\mathcal{A}$ would reach a configuration (q, r) where $r = r'_i$. Since $r'_i a'_{i+1} z_i \sim_R r'_i b_{i+1} \sigma_1(z_i)$, if from (q, r) , $a'_{i+1} z_i$ is accepted, $\mathcal{A}$ also accepts $b_{i+1} \sigma_1(z_i)$.

Next, we prove by induction that $u'_i = u_i$ and $r'_i a'_{i+1} z_i \sim_R r_i w[i+1 \cdots]$ for all $0 \leq i < d$.

For the base case, obviously, when $i = 0$, we have $u'_0 = u_0 = \varepsilon$. Moreover, $a'_1 z_1 = w$. Since $r'_0 = r_0 = M(u'_0) = \varepsilon$, the claim holds.

Now, for the induction step, assume that $u'_\ell = u_\ell$, $r'_\ell \cdot a'_{\ell+1} z_\ell \sim_R r_\ell \cdot w[\ell+1 \cdots]$. We want to prove that the claim also holds at $\ell+1$. Let $(u_\ell, M(u_\ell) \cdot a_{\ell+1}, E_\ell, u_{\ell+1})$ be the transition taken in $\mathcal{H}$ when reading $a_{\ell+1} = w[\ell+1]$ at (u_ℓ, r_ℓ) and $(u'_\ell, M(u'_\ell) \cdot b_{\ell+1}, E'_\ell, u'_{\ell+1})$ be the transition taken in Algorithm 1 when reading $a'_{\ell+1}$ at (u'_ℓ, r'_ℓ) . By induction hypothesis, we know that $u_\ell = u'_\ell$ and $r'_\ell a'_{\ell+1} z_\ell \sim_R$

$r_\ell \cdot w[\ell+1 \dots]$. It follows that $r'_\ell \cdot a'_{\ell+1} \sim_R r_\ell \cdot w[\ell+1] = r_\ell \cdot a_{\ell+1}$ (the prefixes also have the same type). On the other hand, $M(u_\ell) \cdot a_{\ell+1} = r_\ell \cdot a_{\ell+1} \sim_R r'_\ell \cdot a'_{\ell+1} \sim_R r'_\ell \cdot b_{\ell+1}$ hold. So, $M(u_\ell) \cdot a_{\ell+1} \sim_R M(u'_\ell) \cdot b_{\ell+1}$. We then know that the transition $(u_\ell, M(u_\ell) \cdot a_{\ell+1}, E_\ell, u_{\ell+1})$ and the transition $(u'_\ell, M(u'_\ell) \cdot b_{\ell+1}, E'_\ell, u'_{\ell+1})$ are the same transition since $u'_\ell = u_\ell$ and $\mathcal{H}$ is deterministic. Therefore, $u_{\ell+1} = u'_{\ell+1}$. In fact, by Claim (ii), we have $r'_\ell b_{\ell+1} \sigma_1(z_\ell) \sim_R r'_\ell \cdot a'_{\ell+1} \cdot z_\ell$. Since $r'_\ell \cdot a'_{\ell+1} z_\ell \sim_R r_\ell \cdot w[\ell+1 \dots]$, we also have that $r'_\ell b_{\ell+1} \sigma_1(z_\ell) \sim_R r_\ell \cdot w[\ell+1 \dots]$. By removing the letters in the same positions from $r'_\ell \cdot b_{\ell+1}$ and $r_\ell \cdot a_{\ell+1}$, we know that $M(r'_\ell \cdot b_{\ell+1}) \cdot \sigma_1(z_\ell) \sim_R M(r_\ell \cdot a_{\ell+1}) \cdot w[\ell+2 \dots]$. Further, we have $M(r'_\ell \cdot b_{\ell+1}) \cdot \sigma_1(z_\ell) \sim_R \sigma_2(M(r'_\ell \cdot b_{\ell+1}) \cdot \sigma_1(z_\ell)) = M(u'_{\ell+1}) \cdot \sigma_2(\sigma_1(z_\ell)) = r'_{\ell+1} \cdot a'_{\ell+2} z_{\ell+1}$ and $M(r_\ell \cdot a_{\ell+1}) \cdot w[\ell+2 \dots] = r_{\ell+1} \cdot w[\ell+2 \dots]$. Therefore, it holds that $u'_i = u_i$ and $r'_i a'_{i+1} z_i \sim_R r_i \cdot w[i+1]$ for all $0 \leq i < d$.

With similar argument, we can also prove that $u'_d = u_d$. Therefore, we have completed the proof. $\square$

Lemma 15. *Let w be the counterexample provided by the teacher. Algorithm 1 terminates after at most $|w|$ iterations and returns a suffix v such that $ub \in X$ is not $S \uplus \{v\}$ -consistent with a word $u' \in U$ with which ub is currently S -consistent.*

Proof. By Lemma 27, the state sequence over w and that in Algorithm 1 coincide. Therefore, for the last state u_d , we have $\text{MQ}(u_d) \neq mq$. Therefore, the membership results must flip at some point before reaching the last state u_d and the if condition will be satisfied. Hence, Algorithm 1 terminates after at most $|w|$ iterations.

Assume it returns at the i th iteration with $v = \sigma_2(\sigma_1(z_{i-1}))$ and some u'_i such that

$$\text{MQ}(u'_i \cdot v) \neq \text{MQ}(u'_{i-1} \cdot a'_i z_{i-1}),$$

where the one-letter extension $u'_{i-1} b_i$ is currently S -consistent with u'_i . Since

$$\sigma_2(\sigma_1(M(u'_{i-1} a'_i))) = \sigma_2(M(u'_{i-1} b_i)) = M(u'_i),$$

we obtain

$$\sigma_2(\sigma_1(M(u'_{i-1} a'_i))) \cdot \sigma_2(\sigma_1(z_{i-1})) = M(u'_i) \cdot v.$$

Moreover, by proof of Lemma 27,

$$u'_{i-1} a'_i z_{i-1} \in L \iff u'_{i-1} b_i \sigma_1(z_{i-1}) \in L,$$

and hence,

$$\text{MQ}(u'_{i-1} a'_i z_{i-1}) = \text{MQ}(u'_{i-1} b_i \sigma_1(z_{i-1})) = \text{MQ}(\sigma_2(u'_{i-1} b_i) \cdot v)$$

Thus, $\text{MQ}(\sigma_2(u'_{i-1} b_i) \cdot v) \neq \text{MQ}(u'_i \cdot v)$, and $u'_{i-1} b_i$ is not $S \uplus \{v\}$ -consistent with u'_i , witnessed by the word v and the mapping σ_2 . $\square$

We now provide the missing proof for Theorem 16.

Theorem 16. *Let $\mathcal{A}$ be a complete canonical minimal DRA of L with n states and m transitions defined under $\cong_L$, and let d be the maximum length of any counterexample returned by the teacher. It holds that:*

- Algorithm 2 terminates and returns a DRA $\mathcal{H}$ that is isomorphic to $\mathcal{A}$.
- Algorithm 2 terminates after at most $m \times (n - 1)$ equivalence queries.
- Algorithm 2 requires m memorability queries and $\mathcal{O}(d \times m \times n + m^3 \times n^3)$ membership queries.

Proof. When Algorithm 1 returns a distinguishing suffix v separating ub from the state u' with which ub is currently S -consistent, we add v to the suffix set S to correct the transition from u over b . Consequently, ub becomes not $S \uplus \{v\}$ -consistent with u' . Each transition can be corrected at most $n - 1$ times; that is, the target state of a transition over b from u may be misclassified no more than $n - 1$ times, as once ub is inconsistent with a state u'' , it remains inconsistent with u'' thereafter by Lemma 25. Each counterexample will correct at least one transition. Hence, the learner asks at most $m \cdot (n - 1)$ equivalence queries in total. It is worth noting that in our experiments, new states will usually be discovered after adding the suffix returned from the counterexample analysis procedure.

For memorability queries, we only need it for every outgoing transition. The reason is as follows: first, for the initial state ε , $M(\varepsilon)$ is trivially ε , so there is no need to ask memorability queries for the empty word. The only place we need to ask memorability queries is when we obtain a new one-letter extension and add it to the extension set X . (All new states come from X , so their memorable words have been queried when first added into X .) Since the number of all one-letter extensions is equivalent to the number of transitions, the number of memorability queries is m . So, the learner needs m memorability queries.

For membership queries, we need (1) $\mathcal{O}(n \times n \times m)$ queries to fill the table entries since there are at most $m \times n$ counterexamples/suffixes, (2) $\mathcal{O}(d \times n \times m)$ membership queries to analyze the counterexamples where each analysis costs at most d membership queries and we need to analyze counterexamples at most $m \times n$ times, and (3) $\mathcal{O}(m^3 \times n^3)$ membership queries to look for the S -consistency class for each extension in X when constructing the conjectured DRA where $|X| \leq m$, each transition will cost at most $n \times n \times m$ times during a DRA construction with n being the maximum row number and $m \times n$ being maximum column number and we need to construct conjectured DRAs at most $m \times n$ times. Therefore, we need in total $\mathcal{O}(d \times m \times n + m^3 \times n^3)$ membership queries. $\square$

E Preliminary Experiments

We implemented the proposed active learning algorithm in Python⁷ and evaluated it on all examples presented in this paper, as well as on randomly generated DRAs. All experiments were conducted on a machine running Ubuntu 24.04 LTS, equipped with an Intel i7-11800H CPU and 16 GB of RAM.

Query Complexity. Due to space constraints, we report results only for the family of DRAs introduced in Example 2, namely the languages L_n for $n = 1, \dots, 25$. The minimal DRA recognizing L_n has three states and three transitions when $n = 1$, and $2n$ states and $6n - 6$ transitions when $n > 1$. As

⁷ <https://github.com/liyong31/RALearning.git>

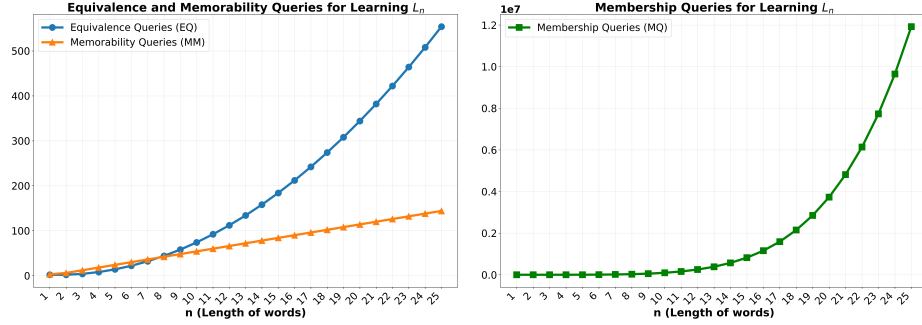


Fig. 5: The x-axis is the the length of word $n = 1, \dots, 25$ for L_n . Left: the number of EQs and Mems for each n . Right: the number of MQs.

shown in Fig. 5, the numbers of all query types grow polynomially with the size of the target DRA (measured by $2n$ states and $6n - 6$ transitions). Among them, the number of MQs grows the fastest, followed by EQs and then Mem queries, in line with the bounds in Theorem 16. Additional results are reported in Section F.

Evaluation of RaLib and our approach. We also conducted experiments on L_n using RaLib [10, 12], using the `iosimulator` backend and default settings on L_n for $2 \leq n \leq 10$, without imposing a time limit. However, a direct comparison is not straightforward: RaLib⁸ targets register extended finite state machines (EFSMs) [12], which incorporate both inputs and outputs, whereas our model does not support outputs. Moreover, EFSMs characterize safety languages and do not explicitly distinguish between accepting and rejecting states.

To enable comparison, we translate our register automata into EFSMs by introducing outputs: `OOK` for transitions that can reach accepting states and `ONO` for transitions reaching a rejecting sink. Since EFSMs separate input and output states, this translation increases the number of states, as reflected in Table 1, where the input models of RaLib are larger. Thus, the comparison with RaLib is not meant as a direct performance evaluation, but as a demonstration that our approach effectively learns models in a complementary setting. While RaLib learns EFSMs using tree queries (TQs) and approximate equivalence checking, our method uses word-based membership queries and exact equivalence checking.

Table 1 summarizes the results. RaLib requires fewer TQs because tree queries compactly encode multiple words, whereas our method operates on individual words for MQs. As a result, our total runtime is dominated by MQs, while RaLib spends more time on EQs. Despite the larger number of MQs, our approach scales predictably with the size of the target model, as also reflected by the linear growth in memorability (Mem) queries. Furthermore, RaLib employs approximate equivalence checking with probabilistic guarantees, resulting in a small and constant number of EQs. Consequently, the learned models produced by

⁸ <https://github.com/LearnLib/ralib>

Table 1: Experimental results for RaLib and our approach on L_n . DRA sizes are reported as numbers of states, and runtimes are given in seconds. All runtimes are total operation times.

Model	Input DRA size		Learned DRA size		#TQ/MQ		#EQ		TQ/MQ Time (s)		EQ Time (s)		Our Mem	
	RaLib	Ours	RaLib	Ours	RaLib	Ours	RaLib	Ours	RaLib	Ours	RaLib	Ours	#Mem	Time
L_2	9	4	8	4	34	39	2	2	<1	<1	2.19	<1	6	<1
L_3	16	6	10	6	49	156	2	4	<1	<1	2.57	<1	12	<1
L_4	22	8	12	8	68	662	2	8	<1	<1	2.91	<1	18	<1
L_5	28	10	14	10	91	2,173	2	14	<1	<1	3.07	<1	24	<1
L_6	34	12	16	12	118	5,885	2	22	<1	<1	3.37	<1	30	<1
L_7	40	14	18	14	149	13,653	2	32	<1	<1	3.59	<1	36	<1
L_8	46	16	20	16	184	28,192	2	44	<1	1.25	3.76	<1	42	<1
L_9	52	18	22	18	223	53,312	2	58	<1	2.58	4.04	<1	48	<1
L_{10}	58	20	24	20	266	94,412	2	74	<1	5.11	3.70	<1	54	<1

RaLib do *not* always accept exactly the same language as the target DRAs, as observed in our experiments. In contrast, our exact equivalence checking ensures correctness but requires more EQs as the model size increases. Moreover, the well-typed register model used in our work appears to enable fast equivalence checking in practice.

We also considered register benchmarks from the RaLib GitHub repository, which contains a collection of models related to those in the Automata Wiki [31]⁹. However, to the best of our knowledge, there is no established translation from EFSMs to our register automata model. To adapt these benchmarks, we combined each EFSM input transition and its corresponding output transition into a single transition. Since our model does not support multiple parameters as in EFSMs, we instead introduced additional states to process parameters sequentially. All states were made accepting. Nevertheless, certain EFSM constructs remain challenging to encode. For example, a state may have transitions such as $I_get(p)$ and $I_put(p)$, where I_get and I_put are input symbols and p is a parameter. In our translation, input action names are abstracted away, which prevents distinguishing such cases based solely on parameters. As a result, this may introduce nondeterminism in the translated models. After excluding non-deterministic instances, we observed that, with few exceptions such as the alternative bit protocol and password models, the models learned by our tool often accept universal languages and thus collapse to a single state. As these cases provide limited insight for evaluation, we omit them from our experimental results.

F Additional Experiments

The Family of Languages L_n . In addition to the figures in the main paper, we report in the following table the number of equivalence queries (EQ), membership queries (MQ) and memorability queries. We also report the total time, which

⁹ <https://automata.cs.ru.nl/>

includes the time for the teacher to solve the three types of queries and the total membership query resolving time to learn each automaton. We can see that the time consumed by resolving membership queries in the teacher dominates runtime in the whole procedure for learning the target automata.

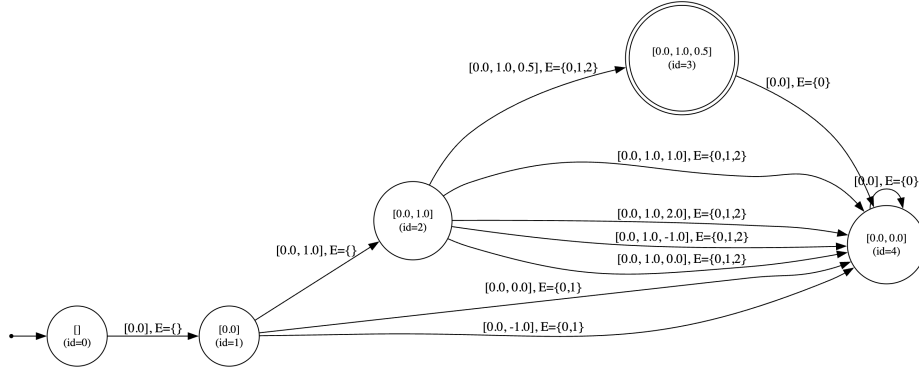
n	States	Transitions	EQ	MQ	Mem	MQ time (s)	Total Time (s)
1	3	3	2	24	3	<1	<1
2	4	6	2	39	6	<1	<1
3	6	12	4	156	12	<1	<1
4	8	18	8	662	18	<1	<1
5	10	24	14	2173	24	<1	<1
6	12	30	22	5885	30	<1	<1
7	14	36	32	13 653	36	<1	<1
8	16	42	44	28 192	42	1.25	1.67
9	18	48	58	53 312	48	2.58	3.23
10	20	54	74	94 412	54	5.11	6.08
11	22	60	92	157 820	60	9.10	10.45
12	24	66	112	250 559	66	15.39	17.32
13	26	72	134	384 197	72	25.21	28.00
14	28	78	158	570 135	78	38.99	42.70
15	30	84	184	822 663	84	59.74	64.97
16	32	90	212	1 155 200	90	88.16	95.11
17	34	96	242	1 590 956	96	129.28	138.45
18	36	102	274	2 148 216	102	181.08	193.26
19	38	108	308	2 852 259	108	256.15	271.12
20	40	114	344	3 730 170	114	347.37	368.70
21	42	120	382	4 814 609	120	473.87	499.51
22	44	126	422	6 133 333	126	615.41	647.93
23	46	132	464	7 730 804	132	823.10	863.34
24	48	138	508	9 644 939	138	1061.34	1111.84
25	50	144	554	11 922 628	144	1369.29	1429.55

The Language L_{mid} . Recall that $L_{mid} = \{a_1 a_2 a_3 \mid a_1 < a_3 < a_2\}$ over $(\mathbb{Z}, <)$. In our experiments, we learn L_{mid} over $(\mathbb{R}, <)$ instead. The minimal well-typed DRA for L_{mid} has 5 states and 11 transitions. Learning this automaton requires 2 EQs, 72 MQs, and 11 memorability queries. The final observation table is shown below.

The DRA learned is shown in Fig. 6. Notice that this figure is generated by our tool, and the position index in E starts from 0 rather than 1.

The Language L_{rep} of Section 4.1. Recall that $L_{rep} = \{xyxy \in \mathbb{Q}^4 \mid x < y\}$. The minimal well-typed DRA for L_{rep} has 6 states and 14 transitions. Learning

	M	ε	$0 \cdot 1 \cdot 0.5$
ε	ε	—	+
0	0	—	—
$0 \cdot 1$	$0 \cdot 1$	—	—
$0 \cdot 1 \cdot 0.5$	ε	+	—
$0 \cdot -1$	ε	—	—

Fig. 6: Learned DRA recognizing L_{mid}

this automaton requires 3 EQs, 156 MQs, and 14 memorability queries. The final observation table and the DRA are the same as the ones in Fig. 2.

Randomly Generated DRAs. We generated random minimal well-typed DRAs of sizes 5, 10, $\dots$, 50, with 50 automata for each size. We report in the following table the **average** (Avg) number of equivalence queries (EQ), membership queries (MQ), and memorability queries and the average time required. As can be seen in Fig. 7, similar to the trend for the family of languages L_n , the number of MQs grows the fastest, compared to EQs and then Mem queries, in accordance with Theorem 16. The average time per automaton here includes the time for the teacher to solve the queries.

Size	Avg #Transitions	Avg #EQ	Avg #MQ	Avg #Mem	Avg Time (s)
5	12.2	2.3	75.0	12.2	0.12
10	34.6	4.4	593.1	34.6	0.28
15	57.4	6.8	2086.1	57.4	1.08
20	82.9	9.2	5131.1	82.9	2.02
25	105.8	11.9	10 542.2	105.8	2.84
30	130.5	13.8	17 883.5	130.5	5.33
35	157.4	15.3	25 949.8	157.4	8.03
40	181.4	18.9	43 782.9	181.4	12.54
45	207.4	20.6	60 489.1	207.4	17.46
50	233.7	22.5	81 354.0	233.7	23.87

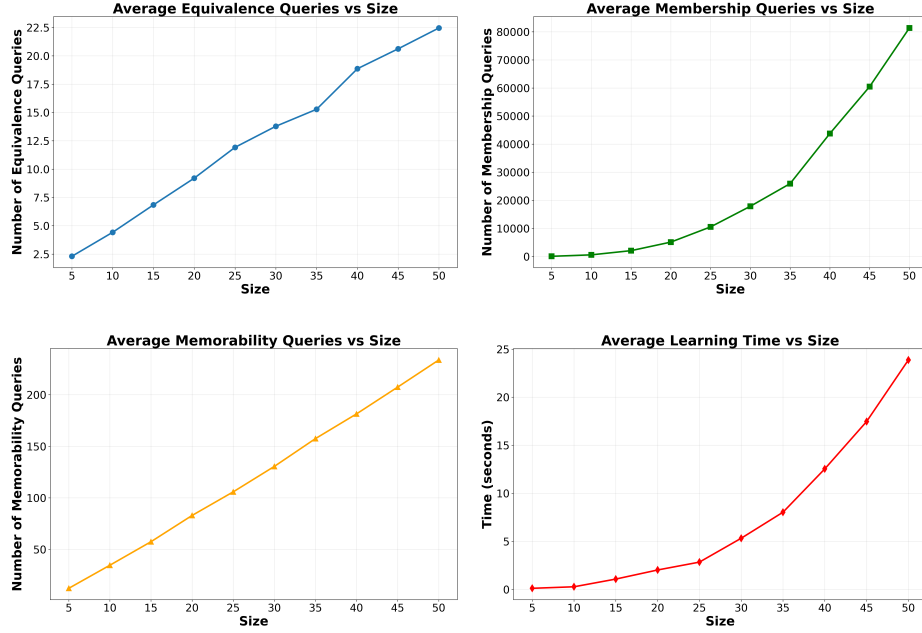


Fig. 7: Average statistics for learning random minimal well-typed DRAs.

G Missing Proofs of Section 4.2

Language Intersection Non-emptiness and Language Inclusion. Given a k_1 -DRA $\mathcal{A}_1 = (Q^1, q_0^1, F^1, \Delta^1)$ and a k_2 -DRA $\mathcal{A}_2 = (Q^2, q_0^2, F^2, \Delta^2)$, the *language intersection non-emptiness* asks whether there is a word $w \in L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$. The *language inclusion* problem asks whether $L(\mathcal{A}_1) \subseteq L(\mathcal{A}_2)$. This is equivalent to the language intersection emptiness problem, that is, whether $L(\mathcal{A}_1) \cap \overline{L(\mathcal{A}_2)} = \emptyset$, where $\overline{L(\mathcal{A}_2)}$ denotes the complement of $L(\mathcal{A}_2)$. Since $\mathcal{A}_2$ is a DRA, it is straightforward to construct a DRA $\mathcal{B}_2$ accepting the complement language, that is, $L(\mathcal{B}_2) = \overline{L(\mathcal{A}_2)}$, by simply swapping the accepting and non-accepting states of $\mathcal{A}_2$.

To solve the language intersection emptiness problem, we can use the product register automaton of Definition 22, which simulates the operations of both automata simultaneously. Recall that this product register automaton differs from the RAs in Definition 1: its registers may contain duplicate values and it is not well-typed. Nevertheless, language non-emptiness can be checked by simply testing whether there exists a path from the initial state (q_0^1, q_0^2) to some accepting state in $F^\times$. Although such a path may be exponentially long—bounded by $|Q_1| \cdot |Q_2| \cdot (k_1 + k_2)!$ —we do not need to store the entire path. It suffices to maintain a counter together with the current configuration of the product

automaton, both of which require only polynomial space. This yields a PSPACE decision procedure.

For the lower-bound, we reduce from the PSPACE-complete problem of deciding whether a linear-bounded deterministic Turing machine M accepts an input w . The reduction closely follows [29, Theorem 32], which considers DRAs that do not permit register permutations. The key is to encode the tape contents using DRA registers, which cannot store duplicate values, and to permute the register symbols while simulating the Turing machine.

Theorem 17. *For DRAs over $(\Sigma, =)$ and over $(\Sigma, <)$, the language intersection non-emptiness problem is PSPACE-complete. Consequently, the language inclusion problem is also PSPACE-complete.*

Proof. Since DRAs can be easily complemented, it suffices to show that the intersection non-emptiness problem for DRAs is PSPACE-complete. The complement problem—the intersection emptiness problem—is also PSPACE-complete. Consequently, the DRA language inclusion problem, which is equivalent to intersection emptiness, is PSPACE-complete as well.

Inclusion in PSPACE. Given a k_1 -DRA $\mathcal{A}_1 = (Q^1, q_0^1, F^1, \Delta^1)$ and a k_2 -DRA $\mathcal{A}_2 = (Q^2, q_0^2, F^2, \Delta^2)$, we ask whether there is a word $w \in L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$. Equivalently, it asks whether there is a run from the initial state (q_0^1, q_0^2) to a final state in $F^\times$ in the product automaton $\mathcal{A}_1 \times \mathcal{A}_2$.

We now consider the product automaton $\mathcal{A}_1 \times \mathcal{A}_2$. We show that if there is a run from the initial state (q_0^1, q_0^2) to a final state in $F^\times$, then there exists a run of length at most $|Q_1| \cdot |Q_2| \cdot (k_1 + k_2)!$ reaching a state in $F^\times$.

Based on this observation, we obtain a nondeterministic polynomial-space algorithm that guesses the input word symbol by symbol. The algorithm keeps only a counter and the current configuration of the product automaton, and therefore uses polynomial space. By Savitch's theorem, PSPACE = NPSPACE, hence the intersection non-emptiness problem for DRAs is in PSPACE.

At a configuration (q^1, q^2, x^1, x^2) , the algorithm guesses a symbol a . The number of possible choices for a is bounded by

$$2(|x^1| + |x^2|) + 1 \leq 2(k_1 + k_2) + 1.$$

To justify this, we reorder the concatenation $x^1 x^2$ into non-decreasing order and retain only the distinct symbols, denoting the resulting string by $x = x_1 \cdots x_\ell$. The guessed symbol a may then be chosen as:

- x_i for any $1 \leq i \leq \ell$,
- a symbol greater than x_ℓ or smaller than x_1 , or
- a symbol strictly between x_i and x_{i+1} for any $1 \leq i \leq \ell - 1$.

After guessing a , the algorithm transitions to the next configuration and increments the counter. It starts with the initial configuration $(q_0^1, q_0^2, \varepsilon, \varepsilon)$ with the counter set to 0, and proceeds as follows:

- If the counter has not yet reached $|Q_1| \cdot |Q_2| \cdot (k_1 + k_2)!$ and a final state is reached, it accepts.
- If the counter has not reached this bound and no final state has been reached, it continues by guessing another symbol.
- If the counter reaches $|Q_1| \cdot |Q_2| \cdot (k_1 + k_2)!$ without reaching a final state, it rejects.

It remains to show that $|Q_1| \cdot |Q_2| \cdot (k_1 + k_2)!$ is an upper bound on the length of a shortest run reaching a final state. Assume that there is a run in the product automaton from the initial state (q_0^1, q_0^2) to a final state in $F^\times$ over a word u of length ℓ . Let the run be

$$\rho_0 = (q_0^1, q_0^2, \varepsilon, \varepsilon) \xrightarrow{u_1} \rho_1 = (q_1^1, q_1^2, x_1^1, x_1^2) \xrightarrow{u_2} \dots \xrightarrow{u_\ell} \rho_\ell = (q_\ell^1, q_\ell^2, x_\ell^1, x_\ell^2),$$

where $(q_\ell^1, q_\ell^2) \in F^\times$.

We say that two configurations

$$\rho_i = (q_i^1, q_i^2, x_i^1, x_i^2) \quad \text{and} \quad \rho_j = (q_j^1, q_j^2, x_j^1, x_j^2)$$

satisfy $\rho_i \approx \rho_j$ if and only if $(q_i^1, q_i^2) = (q_j^1, q_j^2)$ and $x_i^1 x_i^2 \sim_R x_j^1 x_j^2$.

Let $\ell \geq |Q_1| \cdot |Q_2| \cdot (k_1 + k_2)!$. By the pigeonhole principle, there exist two distinct configurations along the run such that $\rho_i \approx \rho_j$. Fix such a pair with $i < j$ so that no earlier–later pair $\rho_k, \rho_{k'}$ with $\rho_k \approx \rho_{k'}$ satisfies either $k \leq i < j < k'$ or $k < i \leq j \leq k'$.

We now construct a new run on a new word that still reaches a final state in $F^\times$, but in which no two configurations are $\approx$ -equivalent. This new run will therefore have length at most $|Q_1| \cdot |Q_2| \cdot (k_1 + k_2)!$.

The idea is to modify the run (and word) from configuration ρ_i onward so that the k -th configuration after ρ_i is ρ'_{j+k} with $\rho'_{j+k} \approx \rho_{j+k}$ for all $0 \leq k \leq (\ell - j)$. Thus the new run is of the form

$$\rho_0 \dots \rho_i = \rho'_j \xrightarrow{u'_{j+1}} \rho'_{j+1} \xrightarrow{u'_{j+2}} \dots \xrightarrow{u'_\ell} \rho'_\ell.$$

It remains to show the following key property: for any transition $\rho_i \xrightarrow{a} \rho_{i+1}$ and any configuration $\rho_j \approx \rho_i$, there exists a symbol a' such that $\rho_j \xrightarrow{a'} \rho_{j+1}$ and $\rho_{j+1} \approx \rho_{i+1}$. Let $\rho_i = (q_i^1, q_i^2, x_i^1, x_i^2)$ and $\rho_j = (q_j^1, q_j^2, x_j^1, x_j^2)$. We only need to pick a symbol a' such that $x_i^1 x_i^2 a \sim_R x_j^1 x_j^2 a'$, which is always possible because $x_i^1 x_i^2 \sim_R x_j^1 x_j^2$ and the data domain is dense.

PSPACE-Hardness. For PSPACE-hardness, we reduce from the PSPACE-complete problem of determining whether a linear bounded deterministic Turing machine M accepts an input w (see, for example, the linear bounded prediction problem in [21]).

More precisely, we consider a Turing machine with binary alphabet and linearly bounded tape $M = \langle Q, q_0, \delta, q_{acc} \rangle$ such that Q is a finite set of states, $q_0 \in Q$ is the initial state, $q_{acc} \in Q$ is the accepting state, and $\delta : Q \times \{0, 1\} \rightarrow$

$Q \times \{0, 1\} \times \{-1, 1\}$ is a transition function. A configuration of M is a triple $\langle q, i, w \rangle$ where $q \in Q$ is the machine state, $0 \leq i < |M|$ is the head position, and $w \in \{0, 1\}^{|M|}$ is the tape content. Let $\delta(q, w(i)) = \langle q', b, j \rangle$. If $1 \leq i + j \leq |M|$, then the state $\langle q', i + j, w[i \mapsto b] \rangle$ is the unique successor of $\langle q, i, w \rangle$. The PSPACE-hard problem asks whether M accepts a word $w_0 \in \{0, 1\}^{|M|}$, that is, whether $\langle q_0, 1, w_0 \rangle$ reaches a configuration with the accepting state q_f .

The main idea of the reduction is similar to the PSPACE-hardness proof of the inclusion problem of a different type of DRAs, the ones where permutations of registers in transitions are not allowed [29, Theorem 32]. Since the values in the register need to be distinct, we construct two $(|M| + 1)$ -DRAs $\mathcal{A}_1, \mathcal{A}_2$ such that whenever they synchronize on a data word, their register assignments ρ_1, ρ_2 represent the content of M tape cells as follows: 0 in the i -th tape cell is represented by $\rho_1(i) = \rho_2(i)$, and 1 by $\rho_1(i) \neq \rho_2(i)$. One extra register plays a technical role that helps maintain the representation. The position of the head and state of the machine are maintained in the state of the automata. We show that M accepts w if and only if $\mathcal{A}_1$ and $\mathcal{A}_2$ both accept some data word w' (intersection non-emptiness).

There are two phases for the DRAs: the *initialization* phase and the *simulation* phase. In the initialization phase, the DRAs are initialized so that they jointly encode the initial configuration of the Turing machine M , with their combined register assignments representing the initial tape contents w_0 .

Initialization Phase. In the initialization phase, both $\mathcal{A}_1$ and $\mathcal{A}_2$ read $2|M|$ symbols, where the $(2i-1)$ -th and $2i$ -th symbols jointly represent $w_0(i)$ for all $1 \leq i \leq |M|$. If $w_0(i) = 0$, both automata verify that these two symbols are identical but distinct from all previously seen symbols, and then store the symbol in their respective registers; otherwise, they transition to rejecting sink states. If $w_0(i) = 1$, the automata ensure that the two symbols are distinct from each other and from all previously seen symbols: $\mathcal{A}_1$ stores the $(2i-1)$ -th symbol and $\mathcal{A}_2$ stores the $2i$ -th symbol. Any violation leads to the rejecting sink.

Thus, whenever neither DRA reaches a rejecting sink, both automata end up in states with register assignments ρ_1 and ρ_2 that together encode the tape contents w_0 .

Each DRA requires $2|M|$ states for this phase, in addition to the initial and rejecting sink states. The extra register may be used to check correct initialization: $\mathcal{A}_2$ first stores the $(2i-1)$ -th symbol, and upon reading the $2i$ -th symbol, if it differs from all previously stored symbols (including the $(2i-1)$ -th), it updates its register accordingly—moving to a new non-sink state as $w_0(i) = 1$ or rejecting otherwise. If the two symbols are equal and new, it interprets this as $w_0(i) = 0$; otherwise, it transitions to a rejecting sink. $\mathcal{A}_1$ can use its extra register in a similar manner.

For example, when $w_0 = 011$, both $\mathcal{A}_1$ and $\mathcal{A}_2$ can read the data word 001234 and end up in (non-sink) states with register assignments $\rho_1 = 013$ and $\rho_2 = 024$ respectively; the data words 001034 and 001134 are rejected by both DRAs and the data word 001232 is rejected by $\mathcal{A}_2$.

Simulation Phase. Once the DRAs are properly initialized, both are in the states representing the initial machine state q_0 and head position 1 (i.e., the head on the first tape cell), with register assignments ρ_1 and ρ_2 that together encode the initial tape contents w_0 . At this point, they are ready to simulate the transitions of M .

Each transition $\delta(q, w(i)) = \langle q', b, j \rangle$, that is, $\langle q, i, w \rangle \rightarrow \langle q', i + j, w[i \mapsto b] \rangle$, is simulated by at most $2|M| + 2$ transitions. Starting at the states representing the machine state q and head position i with register assignments ρ_1 and ρ_2 that jointly encode w , after a sequence of transitions the automata reach the states representing the machine state q' and head position $i + j$, with register assignments ρ'_1 and ρ'_2 that jointly encode the updated tape contents $w[i \mapsto b]$.

The idea is as follows. The automata read two symbols—the i -th symbol in $\mathcal{A}_1$ and the i -th symbol in $\mathcal{A}_2$ —to determine whether $w(i) = 0$ or 1. Based on this information, each automaton simulates the corresponding transition of M . If M does not rewrite $w(i)$, i.e. $b = w(i)$, both automata move to new states representing q' and head position $i + j$ without changing their register assignments, since $w[i \mapsto b] = w$.

Otherwise, when $b \neq w(i)$, the symbol at position i must be updated. Since the register automata can only forget symbols or append new ones at the end of their registers, they first rearrange the registers so that the i -th position can be modified. They do this by reading $2(i - 1)$ symbols— $\rho_1(1) \cdots \rho_1(i - 1)$ and $\rho_2(1) \cdots \rho_2(i - 1)$ —so that the registers are in a form where the i -th entry is ready to be replaced, reaching temporary assignments

$$\begin{aligned} &\rho_1(i) \cdots \rho_1(|M|) \rho_1(1) \cdots \rho_1(i - 1), \text{ and} \\ &\rho_2(i) \cdots \rho_2(|M|) \rho_2(1) \cdots \rho_2(i - 1). \end{aligned}$$

In the case $w(i) = 0$ and it needs to be updated to $b = 1$, that is, $\rho_1(i) = \rho_2(i)$ and the registers must be updated to two different symbols a and b , the automata read and store two fresh symbols, reaching states with updated register assignments

$$\begin{aligned} &\rho_1(i + 1) \cdots \rho_1(|M|) \rho_1(1) \cdots \rho_1(i - 1)a, \text{ and} \\ &\rho_2(i + 1) \cdots \rho_2(|M|) \rho_2(1) \cdots \rho_2(i - 1)b. \end{aligned}$$

Conversely, when $w(i) = 1$ and it needs to be updated to $b = 0$, that is, $\rho_1(i) \neq \rho_2(i)$ and both registers must be updated to the same symbol, the automata read a new symbol a , reaching states with updated register assignments

$$\begin{aligned} &\rho_1(i + 1) \cdots \rho_1(|M|) \rho_1(1) \cdots \rho_1(i - 1)a, \text{ and} \\ &\rho_2(i + 1) \cdots \rho_2(|M|) \rho_2(1) \cdots \rho_2(i - 1)a. \end{aligned}$$

Finally, another $2(|M| - i)$ transitions restore the correct register order, yielding

$$\rho'_1 = \rho_1(i \mapsto a) \quad \text{and either} \quad \rho'_2 = \rho_2(i \mapsto a) \text{ or } \rho'_2 = \rho_2(i \mapsto b).$$

Both automata are in accepting states when they correspond to the machine accepting state q_{acc} . It is easy to see that an accepting computation of M on w_0

corresponds to a data word accepted by both $\mathcal{A}_1$ and $\mathcal{A}_2$, and vice versa. Since both DRAs are of polynomial size in M , the reduction is in polynomial time. Moreover, the DRAs constructed can use identity tests only. $\square$

Configuration Equivalence. Given a DRA $\mathcal{A}$ over (Σ, R) and a configuration (q, v) of $\mathcal{A}$, we define the language of $\mathcal{A}^{(q, v)}$ as $L(\mathcal{A}^{(q, v)}) = \{w \in \Sigma^* \mid \exists q_f \in F, u \in \Sigma^*. (q, v) \xrightarrow{w}_{\mathcal{A}} (q_f, u)\}$. Given two DRAs $\mathcal{A}_1$ and $\mathcal{A}_2$ over (Σ, R) and two configurations (q_1, v_1) and (q_2, v_2) of $\mathcal{A}_1$ and $\mathcal{A}_2$ respectively, the configuration equivalence problem asks whether $L(\mathcal{A}_1^{(q_1, v_1)}) = L(\mathcal{A}_2^{(q_2, v_2)})$.

Language Equivalence. Given two DRAs $\mathcal{A}_1$ and $\mathcal{A}_2$ over (Σ, R) , the language equivalence problem asks whether $L(\mathcal{A}_1) = L(\mathcal{A}_2)$.

The language equivalence problem is a special case of the configuration equivalence problem, that is, it is equivalent to the configuration equivalence problem whether $L(\mathcal{A}_1^{(q_0^1, \varepsilon)}) = L(\mathcal{A}_2^{(q_0^2, \varepsilon)})$ where (q_0^1, ε) and (q_0^2, ε) are the initial configurations of $\mathcal{A}_1$ and $\mathcal{A}_2$ respectively. Both problems are in PSPACE, by arguments analogous to the PSPACE upper-bound proof for language inclusion. Indeed, language equivalence reduces to checking the two inclusions $L(\mathcal{A}_1) \subseteq L(\mathcal{A}_2)$ and $L(\mathcal{A}_2) \subseteq L(\mathcal{A}_1)$, which is then in PSPACE.

Theorem 18. *Both the configuration equivalence problem and the language equivalence problem are in PSPACE for DRAs over dense domain $(\Sigma, <)$.*

Proof. It suffices to prove that the configuration equivalence problem is in PSPACE since the language equivalence problem is a special case of the configuration equivalence problem. Given two DRAs $\mathcal{A}_1$ and $\mathcal{A}_2$ and configurations (q_1, v_1) and (q_2, v_2) of $\mathcal{A}_1$ and $\mathcal{A}_2$, respectively, the configuration equivalence problem $L(\mathcal{A}_1^{(q_1, v_1)}) = L(\mathcal{A}_2^{(q_2, v_2)})$ reduces to checking two inclusions: $L(\mathcal{A}_1^{(q_1, v_1)}) \subseteq L(\mathcal{A}_2^{(q_2, v_2)})$ and $L(\mathcal{A}_1^{(q_1, v_1)}) \supseteq L(\mathcal{A}_2^{(q_2, v_2)})$. Each inclusion further reduces to a language intersection emptiness check, assuming the DRAs start from the given initial configurations. For example, checking $L(\mathcal{A}_1^{(q_1, v_1)}) \subseteq L(\mathcal{A}_2^{(q_2, v_2)})$ reduces to checking $L(\mathcal{A}_1^{(q_1, v_1)}) \cap L(\mathcal{B}_2^{(q_2, v_2)}) = \emptyset$, where $\mathcal{B}_2$ is the complement of $\mathcal{A}_2$, obtained by swapping its accepting and non-accepting states. This intersection emptiness check is in PSPACE, similar to the case of DRAs without initial assignments. $\square$

Memorability. Given a k -DRA $\mathcal{A}$ over $(\Sigma, <)$, a word $u \in \Sigma^*$ and $a \in u$, the memorability problem asks whether a is L -memorable in u , that is, whether $a \in \text{mem}(u)$.

We first present a simpler version of Definition 4 for the case in which the alphabet Σ is dense. In Definition 4, the alphabet Σ may be dense or non-dense. When Σ is non-dense, we may not find a symbol b and an R -preserving mapping σ for u with $\sigma(a) = b$ and $u \sim_R \sigma(u)$, hence the definition refers to another word of the same type. In contrast, for dense Σ , such a symbol b and mapping σ always exist. We simplify the definition of memorable symbol for dense alphabet as follows.

Lemma 28. *Let L be a language over (Σ, R) , where Σ is dense. A symbol $a \in \Sigma$ is L -memorable in a word $u \in \Sigma^*$ if and only if there exist a symbol $b \in \Sigma$, a word $v \in \Sigma^*$ and an R -preserving mapping σ with $\sigma(a) = b$ and $\sigma(d) = d$ for all other $d \in u \cdot v$, such that v is an L -distinguishing extension for u and $\sigma(u)$, that is, either $u \cdot v \in L$ or $\sigma(u) \cdot v \in L$.*

Proof. First, we consider the only-if direction. Let $u' = u$, $a' = a$, $b' = b$. We have (1) $u \cdot a = u' \cdot a' \sim_R \sigma(u' \cdot a')$; and (2) either $u \cdot v = u' \cdot v \in L$ or $u \cdot \sigma(v) = u' \cdot \sigma(v) \in L$.

Now, we consider the if direction. Suppose we have a word u' of the same $\sim_R$ -type as u , symbols $a', b' \in \Sigma$, a word $w \in \Sigma^*$ and an R -preserving mapping σ with $\sigma(a') = b'$ and $\sigma(d) = d$ for all other $d \in u' \cdot w$ such that the two conditions in Definition 4 hold. Since Σ is dense, we can use $\sigma_{u \rightarrow u'}$, the bijective R -preserving mapping for u and u' . Let $b = \sigma_{u \rightarrow u'}^{-1}(b')$ and $v = \sigma_{u \rightarrow u'}^{-1}(w)$. Let σ' be a mapping such that $\sigma'(a) = b$ and $\sigma'(d) = d$ for all other symbols $d \in u \cdot v$. We have

$$u \cdot v \sim_R u' \cdot w \sim_R \sigma(u' \cdot w) \sim_R \sigma_{u \rightarrow u'}^{-1}(\sigma(u' \cdot w)) = \sigma'(u \cdot w) \quad (4)$$

$$u' \cdot w \sim_R \sigma_{u \rightarrow u'}^{-1}(u' \cdot w) = u \cdot v \quad (5)$$

$$\sigma(u') \cdot w \sim_R \sigma_{u \rightarrow u'}^{-1}(\sigma(u') \cdot w) = \sigma'(u) \cdot v \quad (6)$$

The equality $\sigma_{u \rightarrow u'}^{-1}(\sigma(u' \cdot w)) = \sigma'(u \cdot w)$ in 4 holds since σ only changes the symbol a' in $u' \cdot w$ to b' which is then mapped to b by $\sigma_{u \rightarrow u'}^{-1}$, while all other symbols will be mapped to the corresponding symbols in $u \cdot v$ by $\sigma_{u \rightarrow u'}^{-1}$. It follows from $\sigma_{u \rightarrow u'}^{-1}(w)$ that the equality $\sigma_{u \rightarrow u'}^{-1}(\sigma(u') \cdot w) = \sigma'(u) \cdot v$ in 6 holds.

Since we have either $u' \cdot w \in L$ or $\sigma(u') \cdot w \in L$, together with the relations 5 and 6, we obtain that either $u \cdot v \in L$ or $\sigma'(u) \cdot v \notin L$. $\square$

We use the technical lemma below to establish the PSPACE upper bound. Informally, the lemma states that it suffices to replace the symbol a with an arbitrary symbol b , provided that the resulting word remains order-equivalent to u , and then compare the corresponding residual languages.

Lemma 29. *Let $\mathcal{A}$ be a k -DRA over $(\Sigma, <)$, and let $u \in \Sigma^*$ and $a \in u$. Then $a \in \text{mem}_L(u)$ if and only if, for any fresh symbol b and any simple mapping σ such that $\sigma(a) = b$ and $\sigma(d) = d$ for all other symbol $d \in \Sigma$, we have*

$$u \cdot v \in L(\mathcal{A}) \iff \sigma(u) \cdot v \notin L(\mathcal{A}) \quad \text{for some } v \in \Sigma^*.$$

Proof. Given a word $u \in \Sigma^*$ and $a \in u$, assume $a \in \text{mem}_L(u)$.

By Lemma 28, we know there is a fresh symbol $b \notin u$, a word $v \in \Sigma^*$ and an R -preserving mapping σ with $\sigma(a) = b$ and $\sigma(d) = d$ for all other symbols $d \in u \cdot v$ such that v is an L -distinguishing word for u and $\sigma(u)$.

Let $b' \notin u$ and let σ' be a mapping with $\sigma'(a) = b'$ and $\sigma'(d) = d$ for all other symbols $d \in \Sigma$ such that $u \sim_R \sigma'(u)$. We want to prove that there is an L -distinguishing word v' for u and $\sigma'(u)$. If $b' = b$, this is immediate by taking

$v' = v$. For the case $b' \neq b$, we show below that it is always possible to find such a v' .

Without loss of generality assume $b' > a$. We define a mapping σ'' that helps to construct v' from v . The key idea is to map all symbols in v that lie between a and b' to symbols greater than b' , while preserving their relative order with respect to all symbols except b' . Formally, let w be the increasing sequence of symbols in v that satisfy $a < w[i] \leq b'$ for all i , and $w[i] < w[j]$ whenever $i < j$.

Let $c \in u \cdot v$ be the smallest symbol strictly larger than b' such that there is no symbol in $u \cdot v$ between b' and c . We now define σ'' as follows:

$$\sigma''(w[i]) = b' + \frac{c - b'}{|w| + 1} \cdot (i + 1) \quad \text{for all } w[i],$$

and $\sigma''(d) = d$ for all other symbols $d \in v$.

We then let $v' = \sigma''(v)$. It is not hard to verify that

$$u \cdot v \sim_R u \cdot v' \quad \text{and} \quad \sigma(u \cdot v) \sim_R \sigma'(u \cdot v').$$

This completes the proof. $\square$

Theorem 19. *The memorability problem is in PSPACE for DRAs over dense domain $(\Sigma, <)$.*

Proof. Given a DRA $\mathcal{A} = (Q, q_0, F, \Delta)$, a word u , and a symbol $a \in u$, we give a PSPACE algorithm to decide whether $a \in \text{mem}_L(u)$. Let $b \notin u$ be a symbol such that $u \sim_R u'$, where u' is obtained from u by replacing the occurrence of a with b . By Lemma 29, it suffices to check whether there exists an L -distinguishing word v for u and u' .

This reduces to checking whether two configurations (q, r) and (q', r') are equivalent, where

$$(q_0, \varepsilon) \xrightarrow{u}_{\mathcal{A}} (q, r) \quad \text{and} \quad (q_0, \varepsilon) \xrightarrow{u'}_{\mathcal{A}} (q', r').$$

By Theorem 18, configuration equivalence for DRAs can be decided in PSPACE. Thus, whether a is L -memorable in u can also be checked in PSPACE. $\square$